PROJECT TOPICS

Network Security

Assignments Project List · 2026/2027

Manuel E. Correia

Topic overview

#	Project theme	Max groups
1	**Secure Web APIs: OWASP API Security, Zero Trust, Deployment and Testing**	2
2	**Anonymity and Onion Routing**	2
3	**(Non-) Anonymity in Tor**	2
4	**Certificate Revocation in Automated PKI Infrastructures**	2
5	**Ransomware**	2
6	**Intrusion Detection Systems (IDS)**	2
7	**Intrusion Prevention Systems (IPS)**	2
8	**Honeypots & Honeynets for Detecting Intrusions**	2
9	**Increasing Network Security with eBPF**	2
10	**European Digital Identity Wallet — Architecture and Reference Framework**	2
11	**Laboratory: MITM with Ettercap, Credential Exposure and Metasploit Exploitation**	2
12	**Post-Quantum Network Security: TLS, PKI and Crypto-Agility**	2
13	**QUIC / HTTP/3 Security and Encrypted Network Observability**	2
14	**WireGuard Overlay Networks: Tailscale vs NetBird**	2

Conteúdo

Topic overview 1
Common project model 5
1. Secure Web APIs: OWASP API Security, Zero Trust, Deployment and Testing 6
Context and challenge 6
Key concepts 7
Contextual reading 8
Assignment #1 — State of the art and secure API architecture 8
Assignment #2 — Secure Web API proof of concept and evaluation 8
Further / research references 9
2. Anonymity and Onion Routing 10
Context and challenge 10
Key concepts 11
Contextual reading 11
Assignment #1 — State of the art 11
Assignment #2 — Practical work 12
Further / research references 12
3. (Non-) Anonymity in Tor 13
Context and challenge 13
Key concepts 14
Contextual reading 14
Assignment #1 — State of the art 14
Assignment #2 — Practical/measurement work 15
Further / research references 15
4. Certificate Revocation in Automated PKI Infrastructures 16
Context and challenge 16
Key concepts 17
Contextual reading 18
Assignment #1 — State of the art and experimental PKI design 18
Assignment #2 — Automated PKI proof of concept and revocation experiments 18
Further / research references 19
5. Ransomware 20
Context and challenge 20
Key concepts 21
Contextual reading 21
Assignment #1 — State of the art 21
Assignment #2 — Safe simulation 22
Further / research references 22

6. Intrusion Detection Systems (IDS) 23
Context and challenge 23
Key concepts 24
Contextual reading 24
Assignment #1 — State of the art 24
Assignment #2 — Experimental work 25
Further / research references 25
7. Intrusion Prevention Systems (IPS) 26
Context and challenge 26
Key concepts 27
Contextual reading 27
Assignment #1 — State of the art 27
Assignment #2 — Experimental work 28
Further / research references 28
8. Honeypots & Honeynets for Detecting Intrusions 29
Context and challenge 29
Key concepts 30
Contextual reading 30
Assignment #1 — State of the art 30
Assignment #2 — Practical work 31
Further / research references 31
9. Increasing Network Security with eBPF 32
Context and challenge 32
Key concepts 33
Contextual reading 33
Assignment #1 — State of the art 33
Assignment #2 — Prototype and evaluation 34
Further / research references 34
10. The European Digital Identity Wallet Architecture and Reference Framework (ARF) 35
Context and challenge 35
Key concepts 36
Contextual reading 36
Assignment #1 — State of the art 36
Assignment #2 — Pilot/prototype 37
Further / research references 37
11. Laboratory: MITM with Ettercap, Credential Exposure and Metasploit Exploitation 38
Context and challenge 38
Key concepts 39

Contextual reading 39
Assignment #1 - State of the art and laboratory design 40
Assignment #2 - Experimental attack chain and defense evaluation 41
Further / research references 41
12. Post-Quantum Network Security: TLS, PKI and Crypto-Agility 42
Context and challenge 42
Key concepts 43
Contextual reading 43
Assignment #1 — State of the art 43
Assignment #2 — Prototype and measurement 44
Further / research references 44
13. QUIC / HTTP/3 Security and Encrypted Network Observability 45
Context and challenge 45
Key concepts 46
Contextual reading 47
Assignment #1 — State of the art 47
Assignment #2 — Experimental evaluation 47
Further / research references 48
14. WireGuard Overlay Networks: Tailscale vs NetBird 49
Context and challenge 49
Key concepts 50
Contextual reading 50
Assignment #1 — State of the art and comparative architecture 50
Assignment #2 — Comparative deployment and experimental evaluation 51
Further / research references 51
Rules for project submission and evaluation 52
Recommended report structure 52
Submission packaging 52

Common project model

- **Assignment #1 — State of the art.** A focused technical report covering the problem, threat model, relevant mechanisms, current research, limitations and open challenges, with comparison of approaches rather than mere description of tools. It must conclude with an implementation roadmap for Assignment #2, defining the proof-of-concept scope, milestones, milestone deliverables, dependencies, validation criteria and an indicative schedule.

- **Assignment #2 — Practical/experimental extension.** A reproducible proof of concept, prototype, controlled experiment or measurement study derived from Assignment #1. The report must present methodology, results, limitations and countermeasures.

- **Evidence and reproducibility.** Use versioned software, configuration files, scripts, PCAPs/logs or datasets where appropriate, with enough detail for another group to reproduce the experiment.

- **Safety and authorization.** Confine all offensive or potentially disruptive experiments to systems and networks explicitly authorized for the project. Public Internet targets are out of scope unless the instructor explicitly authorizes passive measurement.

1. Secure Web APIs: OWASP API Security, Zero Trust, Deployment and Testing

Capacity	Maximum 2 groups

Architecture-first API security across design, implementation, deployment, testing and operation.

Context and challenge

Modern web APIs are security boundaries through which users, applications, partners and internal services obtain access to data and business capabilities. The OWASP API Security Top 10 (2023) provides a useful taxonomy of recurrent API-specific failure modes, including Broken Object Level Authorization (**BOLA**), Broken Authentication, Broken Object Property Level Authorization (**BOPLA**), Broken Function Level Authorization (**BFLA**), unrestricted access to sensitive business flows, Server-Side Request Forgery (**SSRF**), resource exhaustion, security misconfiguration, improper inventory management and unsafe consumption of third-party APIs. In this project, **OWASP** is used as a threat-identification and verification framework within a broader **API security** architecture spanning *secure design*, *identity and access control*, *zero-trust enforcement*, *deployment*, *testing*, *observability and operational governance*. Students should select and justify the risks that are relevant to their chosen architecture, relate them to concrete assets and trust boundaries, and derive from them explicit security requirements and reproducible verification tests.

The project should adopt a **lifecycle view** of API security. José Haro Peralta's *Secure APIs: Design, Build, and Implement* organizes the problem around design, implementation and infrastructure, and then extends it to identity, secure deployment, observability and testing. This is a useful mental model: an API may be secure at the code level and still be exposed by an unsafe resource model, an authorization gap, a weak deployment topology, leaked secrets, an overly permissive gateway, an undocumented endpoint or insufficient operational visibility. Security should consequently be designed into the API from its specification and threat model through deployment and operation, rather than added as a final penetration-testing activity.

A strong starting point is an **API-first / contract-first** approach. The API contract should explicitly define resources, operations, schemas, validation constraints, error behavior and security schemes before implementation. The OpenAPI Specification 3.1.1 provides a machine-readable description of HTTP APIs and can become a security artefact as well as documentation: it supports inventory, design review, contract testing and automated test generation. A specification does not make an API secure, but undocumented or drifting APIs make it substantially harder to reason about attack surface, access-control policy and test coverage.

Threat modelling should then connect the application's assets and business flows to the relevant OWASP API risks. Authorization deserves particular attention because API clients manipulate object identifiers and invoke operations directly, bypassing the restrictions that a graphical user interface may appear to impose. BOLA, BOPLA and BFLA are therefore not simply coding mistakes: they are failures to define and enforce who may perform which operation on which resource and under which conditions. Sensitive business flows, rate limits, resource consumption, outbound calls and third-party API dependencies should be modelled with the same discipline.

The security architecture must also be explicitly based on **zero-trust principles**. In NIST SP 800-207, network location is not itself a basis for trust: access is granted to resources through explicit policy decisions based on identity and context. Applied to web APIs, this means that an endpoint must not become trusted merely because it is "internal" or reachable from a private subnet. User-to-service and service-to-service requests should be authenticated where appropriate, authorized according to least privilege, validated at every trust boundary, and evaluated independently of network proximity. Zero trust is therefore an architectural model, not a product and not a synonym for input validation.

Identity and access control are a central part of that model. Students should distinguish authentication from authorization and understand the different roles of OAuth 2.0, OpenID Connect and token formats such as JWT. OAuth provides delegated authorization; OpenID Connect adds an identity layer; a JWT is only a representation

format and does not by itself define a secure access-control model. Current deployments should be evaluated against RFC 9700 — Best Current Practice for OAuth 2.0 Security, including the avoidance of deprecated or unsafe patterns. Higher-assurance designs may additionally examine sender-constrained tokens, mTLS/DPoP and the FAPI 2.0 Security Profile, but these mechanisms should be introduced only when justified by the threat model.

Deployment architecture is equally important. NIST SP 800-204 identifies authentication, access management, secure communication, service discovery, monitoring, load balancing, throttling and resilience as core concerns in microservice systems, while NIST SP 800-204A discusses service-mesh-based security infrastructure. A practical API deployment should therefore consider TLS termination, API gateways or reverse proxies, service identities, segmentation, firewall policy, rate limiting, secret management, private versus public endpoints, and reproducible configuration through containers and Infrastructure as Code. Gateways and WAFs can enforce useful controls, but they do not repair broken business authorization or insecure application logic.

Security testing must be derived from the threat model and the API contract. Useful layers include unit and integration tests for access-control decisions, negative tests across user/role/tenant combinations, contract testing, schema and input fuzzing, resource-abuse tests, and business-flow tests. Specification-driven tools are especially relevant: RESTler demonstrated stateful REST API fuzzing from API specifications, and the broader research literature shows that REST API testing remains a substantial research area. The 2023 survey by Golmohammadi, Zhang and Arcuri reviews 92 studies and provides a useful map of automated testing approaches and open challenges. Students should combine automation with targeted tests for authorization and business logic, where generic scanners are often insufficient.

Finally, a deployed API must remain observable and governable. API inventory, version lifecycle, structured logs, traces and metrics are needed to detect undocumented endpoints, access-control failures, abnormal resource consumption and abuse of sensitive business flows. Telemetry must itself be designed securely: credentials, access tokens and sensitive payloads should not be indiscriminately logged. The project should therefore connect prevention (secure design), enforcement (identity and architecture), verification (testing) and detection (observability) into one coherent security model.

The practical challenge is to show that this architecture works. *Students should build a non-trivial web API proof of concept, deliberately introduce a small set of representative OWASP API weaknesses in a controlled baseline, and then demonstrate how architectural and implementation controls remove or reduce the corresponding attack paths*. The result should be an evidence-based comparison between an insecure baseline and a hardened deployment, not a catalogue of tools or screenshots.

Key concepts

- **API contract and attack surface.** The OpenAPI contract defines the intended externally visible behavior of the API and provides a basis for inventory, review, test generation and detection of implementation drift.
- **Security by design and lifecycle security.** Security decisions belong in API design, implementation, deployment, testing and operation. “Shift left” reduces late architectural rework; operational controls and observability provide the corresponding “shield right”.
- **Threat model and OWASP API Security Top 10.** The OWASP list is a risk-awareness taxonomy, not a complete security architecture. Use it to select realistic attack paths and test cases that are justified by the assets, actors and trust boundaries of the chosen application.
- **Authentication, authorization and token security.** Authentication establishes an identity; authorization decides whether that identity may perform a specific action on a specific resource. OAuth, OpenID Connect, scopes/claims, roles and policy models must be used with this distinction in mind.
- **Zero trust for APIs.** Do not grant implicit trust because a request originates inside the network. Apply explicit identity, least privilege and policy enforcement to user-to-service and service-to-service interactions, with resource-centric access decisions.
- **API gateway, service mesh and application controls.** Gateways can centralize TLS, routing, rate limiting and some policy enforcement; service meshes can provide service identity and protected service-to-service communication. Neither replaces application-level authorization and business-logic checks.

- **Secure deployment and configuration.** Network segmentation, secret management, TLS, private endpoints, hardened containers, reproducible Infrastructure as Code, versioned configuration and API inventory are part of the API security posture, not operational afterthoughts.

Contextual reading

Haro Peralta, J. — Secure APIs: Design, Build, and Implement (Manning). — Primary conceptual reference for API security by design, zero-trust APIs, authentication/authorization, secure infrastructure, observability and testing.

OWASP API Security Top 10 — 2023 edition. — API-specific risk taxonomy and mitigation guidance; use it to frame threats, not as a complete architecture.

OWASP — What's Next for Developers / DevSecOps. — Places API security in architecture, requirements, secure SDLC and continuous verification.

OpenAPI Specification 3.1.1. — Machine-readable API contract used for design, documentation, inventory and specification-driven testing.

OpenID Connect Core 1.0 — Normative identity-layer specification; useful for distinguishing OpenID Connect authentication from OAuth 2.0 authorization.

Assignment #1 — State of the art and secure API architecture

- Select a realistic web-API scenario (for example, e-commerce, institutional services, IoT management, collaborative applications or another instructor-approved domain) and define the protected assets, user/service roles, sensitive operations and trust boundaries.
- Produce a threat model that identifies realistic attacker capabilities and maps the most relevant threats to the OWASP API Security Top 10. The goal is depth rather than coverage of all ten items: select a coherent subset of risks that is relevant to the chosen architecture.
- Define an API-first design and an initial OpenAPI 3.1 specification for the most important resources and operations. The specification should include data schemas, validation constraints, security schemes and the operations needed to exercise the selected security properties.
- Design the authentication and authorization architecture. Explain how user and service identities are established, how OAuth 2.0 / OpenID Connect (or another justified standards-based mechanism) is used, how tokens/credentials are validated, and how object-, function- and business-flow-level authorization is enforced.
- Define a zero-trust security model for the API. Identify policy enforcement points, state which interactions require explicit authentication/authorization, show how least privilege is achieved, and explain why internal network location is not treated as sufficient evidence of trust.
- Design the deployment architecture: public and private endpoints, gateway/reverse proxy, service segmentation, TLS, secret management, rate limiting, container/IaC configuration, logging/telemetry and any service-mesh or WAF components that are justified by the threat model.
- Define a security testing strategy that maps each selected threat/control to a reproducible test: access-control matrices and negative tests, contract tests, fuzzing, resource-abuse tests, configuration checks and tests of sensitive business flows. Identify which tests can be automated and which require semantic knowledge of the application.
- Define success criteria and measurable evidence for Assignment #2, including which attack paths should succeed in the vulnerable baseline, which should fail after hardening, and what logs/metrics should be produced.
- Conclude with **a concrete implementation roadmap for Assignment #2**: proof-of-concept scope, milestones, milestone deliverables, dependencies, validation criteria and indicative schedule. The roadmap should explicitly connect implementation tasks to the threat model and test plan.

Assignment #2 — Secure Web API proof of concept and evaluation

- Implement and deploy the proposed API in an isolated laboratory environment. The proof of concept must be sufficiently non-trivial to exercise authorization and trust boundaries (for example, at least two

user/service roles and multiple protected resource operations; multi-service architectures are encouraged when justified).

- Use a reproducible deployment model, preferably containerized, and version the OpenAPI specification, application configuration, security policy, deployment/IaC files and test artefacts together with the code.
- Implement standards-based authentication/authorization and the zero-trust controls proposed in Assignment #1. If an identity provider is used, document its configuration and trust relationships rather than treating it as a black box.
- Create a controlled vulnerable baseline exposing at least two representative OWASP API Security weaknesses. At least one should exercise authorization or identity (e.g., BOLA/BFLA/Broken Authentication) and at least one should exercise a design, deployment or operational dimension (e.g., SSRF, resource consumption, misconfiguration, inventory or unsafe third-party API consumption).
- Demonstrate each weakness safely, record the preconditions and evidence, implement the corresponding countermeasures, and repeat the same tests against the hardened version. Do not claim that passing a generic scanner proves the API secure.
- Automate a meaningful part of the security testing using the API contract where possible. Appropriate techniques may include unit/integration tests, authorization matrices, Schemathesis/RESTler or equivalent fuzzing/contract testing, OWASP REST assessment procedures, and purpose-built business-flow tests.
- Experimentally validate at least one zero-trust property. For example, show that an internal service or client located on the same private network cannot bypass identity or authorization policy solely because of network location, and show the relevant denial/telemetry evidence.
- Instrument the API sufficiently to analyze security events. Collect structured logs and, where appropriate, metrics/traces for failed authorization, rate limiting, suspicious input or business-flow abuse while ensuring that tokens and sensitive data are not unnecessarily logged.
- Compare the vulnerable and hardened systems using the validation criteria defined in Assignment #1. Discuss attack-surface reduction, remaining assumptions, residual risks, operational complexity and any security/usability/performance trade-offs introduced by the controls.

Group allocation: Maximum two groups. If two groups select this theme, they must use different application scenarios and substantially different primary security emphases. A useful split is (i) identity/authorization and object/function-level access control versus (ii) architecture/deployment, business-flow abuse and operational security, but alternative divisions may be approved.

Further / research references

OWASP Cheat Sheet Series. REST Security Cheat Sheet.

OWASP Cheat Sheet Series. REST Assessment Cheat Sheet.

Rose, S., Borchert, O., Mitchell, S., & Connelly, S. NIST SP 800-207: Zero Trust Architecture. NIST, 2020. DOI: 10.6028/NIST.SP.800-207

Chandramouli, R. NIST SP 800-204: Security Strategies for Microservices-based Application Systems. NIST, 2019. DOI: 10.6028/NIST.SP.800-204

Chandramouli, R., & Butcher, Z. NIST SP 800-204A: Building Secure Microservices-based Applications Using Service-Mesh Architecture. NIST, 2020. DOI: 10.6028/NIST.SP.800-204A

Lodderstedt, T., Bradley, J., Labunets, A., & Fett, D. RFC 9700: Best Current Practice for OAuth 2.0 Security. IETF, 2025. DOI: 10.17487/RFC9700

Golmohammadi, A., Zhang, M., & Arcuri, A. “Testing RESTful APIs: A Survey.” ACM Transactions on Software Engineering and Methodology 33(1), 27:1–27:41, 2024. DOI: 10.1145/3617175

Atlidakis, V., Godefroid, P., & Polishchuk, M. “RESTler: Stateful REST API Fuzzing.” ICSE 2019, pp. 748–758. DOI: 10.1109/ICSE.2019.00083

Santos Filho, A., Rodríguez, R. J., & Feitosa, E. L. “Automated broken object-level authorization attack detection in REST APIs through OpenAPI to colored Petri nets transformation.” International Journal of Information Security 24, Article 83 (2025). DOI: 10.1007/s10207-024-00970-5

OpenID Foundation. FAPI 2.0 Security Profile — Final Specification, February 2025. Official specification

2. Anonymity and Onion Routing

Capacity	Maximum 2 groups

Anonymous-communication overlays: Tor and I2P architectures, adversary models, path construction, service publication, metadata leakage and performance/privacy trade-offs.

Context and challenge

Anonymous communication is not a single property but a family of goals—sender anonymity, receiver anonymity, unlinkability and location privacy—whose strength depends on the observer. Systems such as Tor and I2P build encrypted overlay paths on top of the Internet so that no single relay should learn both endpoints of a communication. The project should begin by defining the anonymity property and adversary model before discussing mechanisms or performance.

Tor is a low-latency anonymity network designed for interactive applications. Its architecture combines directory authorities and consensus, guard/middle/exit relays, layered circuit cryptography and stream multiplexing. The seminal Tor: The Second-Generation Onion Router paper explains the usability/anonymity trade-off behind low-latency routing; the living Tor specifications should be used to understand the current protocol rather than assuming the 2004 design is unchanged.

Onion Services extend this model to service-location privacy. A service publishes descriptors through the Tor distributed directory and uses introduction and rendezvous points so that client and service meet through Tor without directly revealing the service address. The current Rendezvous Specification is therefore important for distinguishing ordinary exit traffic from service-to-service communication that remains inside the overlay.

I2P provides a useful architectural contrast. It is primarily an internal anonymity overlay, relies on unidirectional inbound and outbound tunnels, distributes router and destination information through a network database, and uses techniques generally described as garlic routing. Comparing the systems should therefore cover path directionality, peer discovery, service naming, transport diversity and how each architecture deals with churn, peer trust and censorship.

Layered encryption hides content from intermediate relays but does not erase metadata. Timing, packet direction, volume, connection duration, endpoint behaviour and application identifiers can still leak information. Exit relays also see application plaintext whenever the application protocol itself is not end-to-end encrypted. Students should explicitly separate what the anonymity overlay protects from what must still be protected by TLS, application design and user operational security.

The central scientific trade-off is that stronger traffic-analysis resistance is expensive. Padding, cover traffic, longer or more diverse paths and deliberate delays can improve resistance to inference, but increase latency and bandwidth overhead and may harm usability. The report should therefore compare security claims together with measurable system costs rather than presenting anonymity as a binary property.

A useful way to reason about anonymity systems is to separate the protocol threat model from the stronger adversaries that are often implicitly assumed in discussions. Low-latency networks such as Tor deliberately optimize for interactive use and therefore cannot add arbitrary delay or cover traffic. They provide meaningful protection against many local and partial observers, but were never designed to make traffic correlation impossible for a global passive observer with sufficiently broad visibility. Students should state clearly which observer is being considered and avoid claims such as “Tor makes a user anonymous” without specifying against whom.

Path selection is a security mechanism, not only a routing algorithm. Tor uses long-lived entry guards to reduce the probability that repeated circuits eventually expose the client to a malicious first hop, and uses directory consensus to provide a common view of relay identity and capabilities. The current Tor path-selection specification should be studied together with the architecture because relay weighting, guard persistence, exit policy and churn influence both performance and exposure to adversarial relays. I2P makes different choices around peer profiling and tunnel construction, which creates a valuable comparison of trust and path diversity.

Censorship resistance adds another layer of engineering. Public relay lists can themselves become blocking inputs, so Tor uses bridges and pluggable transports to make access harder to enumerate or fingerprint. The Tor bridge documentation is a useful entry point for understanding that anonymity and reachability are distinct problems: a system may provide strong path privacy once connected yet remain unusable if a censor can reliably block entry. Students should therefore include discoverability, transport fingerprinting and availability in their comparison, without performing experiments against networks they do not control.

The experimental component should treat anonymity as a set of measurable consequences rather than a binary label. Path-establishment latency, throughput, relay/path changes, observable metadata and the cost of a selected privacy mechanism should be measured across repeated runs. The results should be interpreted against the stated threat model: poor throughput is not evidence of stronger anonymity, and successful encrypted communication is not evidence that traffic-analysis resistance is adequate. The Tor Research portal should be used to identify recent work that tests these trade-offs under realistic assumptions.

Key concepts

- **Anonymity and unlinkability.** Anonymity asks who among a set could have performed an action; unlinkability asks whether two actions or identities can be correlated. Both are relative to an adversary model.
- **Low-latency anonymity.** Tor and I2P aim to remain usable for interactive traffic. Low latency preserves timing structure, which creates a fundamental traffic-analysis trade-off absent from high-latency mix networks.
- **Onion routing.** A client establishes a multi-relay path and applies layered cryptographic protection so that each relay learns only the predecessor, successor and state needed to forward traffic.
- **Garlic routing.** In I2P, multiple messages may be bundled and protected together, while inbound and outbound tunnels are built independently; the resulting routing model differs materially from Tor circuits.
- **Directory / network database.** Tor uses signed directory consensus; I2P uses a distributed network database. These mechanisms affect discovery, trust, resistance to malicious routing information and scalability.
- **Onion Service / I2P destination.** Service publication can hide the network location of a server, but does not automatically protect application identifiers, logging practices or administrator OPSEC.
- **Guard and peer selection.** Path-selection rules attempt to limit exposure to malicious relays and repeated observations; they are part of the anonymity design, not merely load balancing.
- **Metadata leakage.** Encryption protects payloads but observable traffic shape, timing and endpoints can still support inference; application-layer encryption and disciplined operation remain necessary.

Contextual reading

Tor Specifications — Living protocol specifications for circuits, relay protocol, directory operation and current cryptographic mechanisms.

I2P technical documentation — Current architecture, transport and NetDB documentation.

Tor Research — Curated gateway to current and historical research relevant to Tor security, performance and censorship resistance.

Tor Browser Manual — Circumvention — Practical introduction to bridges and pluggable transports, complementing the protocol specifications.

Assignment #1 — State of the art

- Choose either Tor or I2P as the primary system and define the anonymity properties and adversary capabilities being considered. The report must distinguish local passive observers, malicious relays/peers, colluding relays, active attackers and more powerful multi-vantage observers.
- Describe the current architecture in sufficient technical depth: node/relay roles, path construction, key establishment, directory or peer discovery, transport protocols, naming/addressing and service publication. Use current specifications rather than only historical tutorials.
- Analyse how cryptographic layering and routing design limit information available to each intermediary, and identify information that remains exposed at the network and application layers.

- Compare the primary system with the alternative (Tor vs I2P) along at least: path directionality, discovery/trust model, service publication, censorship resistance, NAT/firewall handling, latency and bandwidth overhead, and operational complexity.
- Identify at least two open research or engineering challenges—e.g., traffic analysis, malicious relay selection, congestion, denial of service, scalability, bridge/pluggable-transport discovery or anonymous service abuse—and discuss the state of proposed mitigations.
- Define the Assignment #2 roadmap: controlled topology, software versions, measurements, milestones and deliverables, and explicit criteria for what evidence would support or falsify the chosen claims.

Assignment #2 — Practical work

Group allocation Maximum two groups. If two groups select the theme, one should focus on Tor and the other on I2P, unless an alternative division is approved.

- Deploy the selected anonymity system in a controlled environment and expose a small service appropriate to it. Do not target or instrument third-party anonymous users or services.
- Document path establishment and service reachability using client-visible diagnostics and non-sensitive packet captures. Identify which properties are visible on the local network, at an entry point, and at the application endpoint.
- Measure at least connection establishment time, end-to-end latency and transfer throughput across repeated runs; report variability rather than a single measurement.
- Evaluate one privacy-related operational mechanism—such as padding, bridge/pluggable-transport use, path persistence or service isolation—where feasible, and quantify the cost/benefit under the project threat model.
- Demonstrate the difference between overlay anonymity and end-to-end application confidentiality by explaining what an exit or destination can learn with and without TLS or equivalent application encryption.
- Deliver configuration files, measurement scripts and a short reproducibility guide, with any addresses/identifiers that could expose third parties removed from published artefacts.

Further / research references

Dingledine, Mathewson & Syverson, Tor: The Second-Generation Onion Router — USENIX Security 2004; foundational and highly cited Tor paper.

Tor Project, Rendezvous Specification — Current Onion Service protocol.

I2P Project, Garlic Routing — Detailed explanation of I2P routing terminology and design.

3. (Non-) Anonymity in Tor

Capacity	Maximum 2 groups

Limits of Tor anonymity: traffic correlation, website fingerprinting, defenses, operational-security failures and realistic experimental evaluation.

Context and challenge

Tor is designed to make straightforward source tracing difficult, but it does not make all observable information disappear. Deanonymization research asks a more precise question: under a specified adversary model, what observable features remain correlated with a user, service or activity, and how reliably can those features be exploited? This distinction is essential because a protocol can provide its intended cryptographic guarantees while anonymity still fails through traffic analysis, endpoints or operational mistakes.

Website fingerprinting (WF) illustrates the problem well. A local passive observer cannot read TLS or Tor payloads, yet can observe packet direction, timing, burst structure and transferred volume. The influential Deep Fingerprinting work showed that deep learning can exploit these features under controlled assumptions; Tik-Tok demonstrated the value of timing information. These papers are important, but their reported accuracy must be interpreted together with dataset construction and threat-model assumptions.

A major methodological issue is the difference between closed-world and realistic open-world evaluation. Online Website Fingerprinting (USENIX Security 2022) evaluated genuine Tor traffic and showed why monitored-set size, class priors, false positives and temporal drift can radically change practical conclusions. A project on Tor deanonymization should therefore report precision, recall, false-positive rate and base-rate assumptions, not just overall accuracy.

Traffic correlation is broader than website fingerprinting. An adversary with visibility at two or more network positions may attempt to match flows from timing and volume patterns; feasibility depends on vantage points, path churn, congestion and how much traffic is observed. Defenses such as padding, adaptive padding, traffic shaping and cover traffic reduce information leakage at the cost of bandwidth and latency. The 2024 survey A comprehensive analysis of website fingerprinting defenses on Tor is a useful map of this attack/defense trade space.

Real investigations also show that protocol-level attacks are only one source of failure. Onion Services in the Wild (PoPETs 2024) analyzed 136 court cases and found operational mistakes and endpoint-level information to be major deanonymization avenues. Reused identifiers, logging, compromised endpoints, misconfigured services, browser/application behaviour and cross-service identity linkage can defeat anonymity without cryptographically breaking Tor.

For this coursework, all experimentation must be ethical and controlled. Students may use self-generated Tor traffic, public research datasets or instructor-provided traces. The objective is to reproduce or critically evaluate a published result at reduced scale and to test a defense under the same methodology—not to identify real users, operators or third-party Onion Services.

Deanonymization techniques should be categorized before they are compared. Website fingerprinting is usually a single-vantage classification problem; end-to-end traffic correlation assumes observations at two network positions; active attacks deliberately perturb traffic or routing; and endpoint/operational-security failures exploit information outside the anonymity protocol. These categories have different preconditions, costs and countermeasures. A strong report should therefore state precisely what the attacker can observe and manipulate, rather than grouping all techniques under the vague label “Tor attacks.”

Machine-learning results require unusually careful methodology in this field. The 2023 survey The rise of website fingerprinting on Tor identifies unrealistic assumptions that recur in the literature, including closed-world evaluation, isolated page loads, static content and limited temporal variation. Training/test leakage is particularly damaging because traces collected from the same site version, network conditions or time period can make a classifier appear much more robust than it is. Students should use temporally or operationally separated data where possible and report confidence intervals, class priors and false positives.

Defenses should be evaluated as engineering mechanisms, not just as decreases in classifier accuracy. Padding, burst shaping, cover traffic and application-level transformations consume bandwidth, introduce latency or require endpoint changes. The 2024 survey of website-fingerprinting defenses documents these trade-offs and re-evaluates several proposals under improved experimental assumptions; recent work such as Practical and lightweight defense against website fingerprinting illustrates the continuing attempt to reduce overhead. A project should report both privacy gain and cost.

The most defensible experimental goal is reproduction and critical evaluation rather than "deanonymization." Students can generate their own traces, use published datasets, reproduce a classifier or correlation metric, and then test one countermeasure under the same conditions. No experiment should identify real Tor users or enumerate third-party Onion Services. This scope keeps the project scientifically useful while forcing the group to confront a core lesson of privacy research: performance on a curated dataset is only meaningful when the deployment assumptions that generated that dataset are made explicit.

Key concepts

- **Closed world vs open world.** A closed-world classifier assumes every trace belongs to a monitored set; an open-world experiment includes unmonitored traffic and therefore exposes base-rate and false-positive effects.
- **Website fingerprinting.** Inference of the visited service from encrypted traffic features such as packet direction, size, bursts and timing rather than payload contents.
- **Traffic correlation.** Matching flows observed at distinct network locations using statistical similarity in timing and volume; feasibility depends strongly on observer placement and network noise.
- **Base rate.** In real deployments, monitored activities may be rare. Even a classifier with high laboratory accuracy can produce mostly false alarms when the prior probability is low.
- **Concept / temporal drift.** Website content, Tor routing and network conditions change over time; models trained on old traces may lose accuracy.
- **Traffic-analysis defense.** Padding and shaping attempt to reduce distinguishability by altering observable traffic patterns, normally at a measurable latency or bandwidth cost.
- **Protocol security vs endpoint/OPSEC security.** The cryptographic overlay can operate correctly while application identifiers, logs, browser behaviour or operator mistakes reveal identity.
- **Reproducible evaluation.** Report dataset provenance, train/test separation, class priors, feature extraction, random seeds and confidence intervals; avoid conclusions based only on a single accuracy value.

Contextual reading

A comprehensive analysis of website fingerprinting defenses on Tor (Computers & Security 2024) — Survey and comparative discussion of defense mechanisms and overheads.

Tor Research — Curated Tor research references and project guidance.

The rise of website fingerprinting on Tor: Analysis on techniques and assumptions — JNCA 2023 survey emphasizing threat-model assumptions and real-world validity.

Cherubin, Jansen & Troncoso (2022) — Online Website Fingerprinting — USENIX Security study emphasizing realistic online/open-world evaluation and the operational meaning of false positives.

Assignment #1 — State of the art

- Select one peer-reviewed deanonymization or traffic-analysis paper as the anchor study. No two groups should use the same anchor paper without approval.
- Reconstruct the exact threat model: what the adversary can observe/control, whether it is passive or active, what knowledge it has, and which anonymity property is being attacked.
- Explain the information features and statistical/ML method at a conceptual level, then analyse the experimental dataset, closed/open-world assumptions, train/test split, metrics and potential sources of leakage or bias.
- Compare the anchor paper with at least two later papers or defenses. Discuss whether later work changes the practicality, scalability or assumptions of the original result.

- Include a separate section on non-protocol deanonymization: endpoint compromise, identifiers, server logs, application behaviour and OPSEC. Explain why these mechanisms must not be conflated with a break of Tor cryptography.
- Define a safe Assignment #2 roadmap using self-generated/public data: milestones for dataset preparation, baseline reproduction, defense experiment, metric analysis and reproducibility packaging.

Assignment #2 — Practical/measurement work

Ethical constraint Do not attempt to deanonymize real Tor users, third-party Onion Services or operators. Experiments must use self-generated traffic, controlled services or public research datasets.

Group allocation Maximum two groups; no two groups may select the same anchor paper/attack without approval.

- Reproduce a bounded component of the selected attack using public or self-generated traces. Do not collect traffic from uninvolved Tor users or real Onion Services.
- Evaluate the baseline in both a controlled setting and a more realistic setting where feasible (e.g., larger unmonitored set, temporal split or background traffic). Report precision/recall, false-positive rate and confidence intervals.
- Implement or simulate one defense from the literature, such as padding or shaping, and repeat the evaluation under identical conditions.
- Quantify the security/performance trade-off: change in attack effectiveness versus bandwidth overhead, latency or other operational cost.
- Perform at least one robustness check—e.g., retraining on a different time window, changing circuit conditions, or evaluating class imbalance—to show whether conclusions depend on a narrow experimental setup.
- Deliver scripts, parameters and provenance information sufficient for another group to reproduce the experiment without interacting with third-party Tor users.

Further / research references

Sirinam et al., Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning — ACM CCS 2018.

Rahman et al., Tik-Tok: The Utility of Packet Timing in Website Fingerprinting Attacks — PoPETs 2020.

Cherubin, Jansen & Troncoso, Online Website Fingerprinting — USENIX Security 2022 Distinguished Paper.

Tippe & Tippe, Onion Services in the Wild — PoPETs 2024.

Practical and lightweight defense against website fingerprinting — Computer Communications 2024; recent defense with explicit overhead evaluation.

4. Certificate Revocation in Automated PKI Infrastructures

Capacity	Maximum 2 groups

Offline root of trust, online Intermediate CA, ACME lifecycle automation, CRL/OCSP/stapling, short-lived certificates, and relying-party enforcement.

Context and challenge

Certificate revocation addresses a deceptively simple security question: how can a certificate that is still cryptographically valid and has not yet expired be made unacceptable to relying parties? This situation arises after private-key compromise, certificate mis-issuance, loss of authorization, or decommissioning. In an X.509 PKI, the CA's decision to revoke a certificate is only the first step. The security property that matters is **revocation enforcement latency**: how quickly authenticated status information is produced, disseminated, cached and actually enforced by clients. RFC 5280 specifies Certificate Revocation Lists (CRLs), while RFC 6960 specifies OCSP. The end-to-end problem is therefore not merely a PKI data-structure problem; it is a distributed-systems problem involving freshness, availability, privacy, scalability, trust and failure semantics.

The project should use a realistic private-PKI hierarchy as an experimental instrument. The recommended architecture is an **offline Root CA** that is brought online only for exceptional root-level operations, together with an **online Intermediate CA implemented with OpenBao or HashiCorp Vault**. Both OpenBao PKI guidance and Vault PKI guidance recommend keeping the root outside the networked PKI service and installing a root-signed intermediate for operational issuance. This separation reduces exposure of the long-lived root key while allowing the intermediate to support automated day-to-day issuance, renewal and revocation.

ACME (RFC 8555) provides the automation layer for the certificate lifecycle. It standardizes account management, identifier validation, certificate issuance and a **revokeCert** operation, and it enables clients such as cert-manager, Certbot, Caddy or lego to renew certificates without manual operator intervention. However, ACME does **not** solve the relying-party revocation problem: after a CA accepts a revocation request, clients still need a fresh and authentic way to learn the new status and a policy for what to do when status information is unavailable. This distinction — between *revocation initiation* and *revocation status dissemination/enforcement* — is central to the project. Vault and OpenBao expose ACME issuance together with CRL/OCSP facilities, making them particularly suitable for a controlled experiment.

CRLs implement a push/cache model: the CA periodically publishes a signed snapshot of revoked serial numbers and relying parties download and cache it. The mechanism is simple and privacy-preserving at query time, but the **thisUpdate/nextUpdate** interval creates an intrinsic freshness window, and large revocation sets increase bandwidth and processing cost. Delta CRLs reduce update size by distributing changes since a base CRL, but introduce additional state and synchronization requirements. In the laboratory, CRL rebuild frequency, client cache behavior and the difference between revocation time and CRL publication time should be treated as measurable variables rather than implementation details.

OCSP uses a pull model in which a client asks a responder for the status of a specific certificate. It can provide fresher status with small responses, but it introduces an online dependency and can leak which certificate — and therefore often which site — a client is validating. OCSP stapling moves the query to the TLS server, which periodically obtains a signed status response and presents it during the handshake; TLS Must-Staple (RFC 7633) can make the presence of stapled status a client-enforceable requirement. These mechanisms expose the classic **soft-fail versus hard-fail** trade-off: continuing when status is unavailable improves availability but weakens revocation guarantees; failing closed preserves the guarantee but lets a status-service outage deny access to otherwise legitimate services. The operational direction of the public Web PKI is also changing: Let's Encrypt ended its OCSP service in August 2025 and now publishes revocation information through CRLs, explicitly citing privacy and operational complexity among the reasons.

Modern browsers further complicate the textbook model in which every client simply downloads a CRL or performs a live OCSP query. Browser vendors use ecosystem-specific mechanisms and local policy. CRLite is a particularly important example: the original CRLite paper (IEEE S&P 2017) showed that Web-PKI revocation data could be compressed and pushed to browsers for private local checking. More recent work on Clubcards for the

WebPKI and the SIGCOMM 2026 deployment study documents the evolution of CRLite into a production-scale Firefox mechanism. Students should therefore distinguish normative PKIX mechanisms from actual relying-party behavior and empirically verify what different clients do with a private CA rather than assuming browser behavior from the RFCs.

A second line of defence is to shorten certificate lifetime so that compromised credentials expire quickly even if explicit revocation is slow or unavailable. RFC 8739 formalizes ACME STAR — short-term, automatically renewed certificates — as an alternative model in which stopping renewal replaces conventional revocation for the short-term certificate sequence. The Web PKI is also moving toward shorter lifetimes: CA/Browser Forum Ballot SC081v3 introduced a schedule that reduces maximum public TLS subscriber-certificate validity from 398 days to 200 days in March 2026, 100 days in March 2027 and 47 days in March 2029. Private PKIs can experiment with much shorter lifetimes (hours or minutes), but the trade-off is fundamental: shorter lifetimes reduce the maximum compromise window while increasing renewal frequency and dependence on the availability and correctness of the CA, ACME service, DNS/challenge path and deployment automation.

The scientific objective is to quantify these trade-offs. The project should compare explicit revocation mechanisms with short-lived, automatically renewed credentials and answer questions such as: **How long after T0 (revocation) does each relying party actually reject the certificate?** What happens when CRL/OCSP infrastructure is unavailable? How does stapling alter dependencies? At what certificate lifetime does passive expiry become an acceptable substitute for active revocation in a given threat model? What happens when the CA or ACME renewal path is unavailable? A useful measurement model is to record T0, CRL publication time, OCSP-status change time, stapled-response refresh time and the first client rejection. The gap between these events is the practical security exposure window.

A suitable laboratory topology is therefore: an offline Root CA signs an Intermediate CA whose private key remains protected inside OpenBao/Vault; that Intermediate exposes ACME for automated leaf issuance and the configured CRL/OCSP endpoints; an ACME client (for example cert-manager in Kubernetes, or Certbot/Caddy/lego in a simpler environment) installs certificates into one or more TLS workloads; and several relying-party clients are instrumented to test acceptance. The PKI platform is not the research objective by itself — it is the controlled infrastructure needed to make revocation delay, failure semantics and certificate-lifetime trade-offs experimentally observable.

Key concepts

- **Offline Root CA / online Intermediate CA.** The root is the long-lived trust anchor and should have minimal online exposure. The operational intermediate performs routine issuance and revocation and can be rotated without replacing the root trust anchor.
- **Revocation initiation vs. revocation enforcement.** Marking a certificate revoked at the CA does not immediately protect relying parties. Security depends on dissemination, freshness, caching and client policy.
- **CRL and Delta CRL.** A CRL is a signed snapshot of revoked certificates; a Delta CRL carries changes relative to a base CRL. Publication frequency determines how stale a client view can become.
- **OCSP.** A responder returns signed status for a specific certificate. It can improve freshness but creates an online availability dependency and potential privacy leakage.
- **OCSP stapling / Must-Staple.** Stapling shifts status retrieval from the client to the TLS server; Must-Staple can require a valid stapled response. Both change where availability and freshness failures occur.
- **Soft fail / hard fail.** A relying party must decide whether unavailable revocation status is a reason to reject a connection. This policy is a security-versus-availability decision, not merely a protocol detail.
- **ACME lifecycle automation.** ACME automates validation, issuance, renewal and can submit revocation requests, but it does not define how every relying party receives or enforces revocation status.
- **Short-lived certificates / passive revocation.** Short validity bounds the compromise window and can reduce dependence on explicit revocation, but it shifts security toward highly reliable automated renewal and rapid credential replacement.
- **Browser-specific revocation mechanisms.** Modern browsers may use pushed or vendor-curated revocation data rather than generic live OCSP/CRL behavior. CRLite is a prominent production example.

- **Revocation enforcement latency.** The experimentally relevant metric is the time between the decision to revoke and the moment each relying party ceases to accept the compromised certificate.

Contextual reading

Kwon, H., Kim, D., Kim, H., Hahn, C., & Hur, J. (2026). "Certificate Revocation in the TLS Ecosystem: A Survey." ACM Computing Surveys 58(7), Article 178. DOI: 10.1145/3785653 — Current systematic survey comparing revocation schemes in terms of security, efficiency and deployability.

Let's Encrypt (2025) — OCSP Service Has Reached End of Life. Official article — Operational case study for the move away from live OCSP in a major public CA.

CA/Browser Forum, Ballot SC081v3 — Introduce Schedule of Reducing Validity and Data Reuse Periods. Official ballot — Current Web-PKI policy trend toward shorter certificate lifetimes and stronger lifecycle automation.

OpenBao — PKI Secrets Engine considerations. Official documentation — Stable guidance for CA hierarchy, offline root/online intermediate, ACME, CRL/OCSP and operational constraints.

HashiCorp Vault — PKI Secrets Engine and ACME. PKI engine; ACME documentation — Alternative implementation reference for the same laboratory architecture.

Mozilla — "Fast, private and secure (pick three): Introducing CRLite in Firefox." Mozilla Firefox article — Accessible description of production CRLite deployment and the privacy/performance motivation for local revocation checking.

Assignment #1 — State of the art and experimental PKI design

- Explain the trust and key-management model of a hierarchical X.509 PKI and justify the separation between an offline Root CA and an online Intermediate CA. Include the consequences of compromise at the leaf, ACME-account, intermediate and root levels.
- Analyse CRLs, Delta CRLs, OCSP, OCSP stapling/Must-Staple, short-lived certificates and browser-pushed mechanisms such as CRLite. Compare them using explicit criteria: freshness, privacy, bandwidth, availability, scalability, client support, operational complexity and failure semantics.
- Distinguish certificate revocation at the CA from revocation-status dissemination and relying-party enforcement. Explain what ACME automates — including the revokeCert operation — and what remains outside ACME.
- Review current public-Web-PKI practice, including the deprecation of Let's Encrypt OCSP, the deployment of CRLite in Firefox and the CA/Browser Forum trajectory toward shorter certificate validity. Contrast this with private/enterprise PKIs where clients and policy can be controlled more directly.
- Design the practical PKI architecture for Assignment #2: offline Root CA; OpenBao or HashiCorp Vault as an online Intermediate CA; ACME issuance; CRL and OCSP endpoints; at least one TLS workload; and at least two relying-party clients with different revocation-checking behavior.
- Define three experimental strategies: (A) a longer-lived certificate with CRL checking, (B) a comparable certificate with OCSP and, where supported, stapling, and (C) short-lived certificates with automatic ACME renewal. Lab lifetimes may be compressed to minutes/hours to make experiments tractable, but the report must explain how the measurements relate to realistic operational lifetimes.
- Define measurable events and success criteria before implementation. At minimum record T0 (revocation decision), CRL publication/rebuild time, OCSP status-change time, stapled-response refresh time where applicable, and first rejection by each client. Also define availability experiments for CRL/OCSP outage and CA/ACME renewal outage.
- Conclude with the implementation roadmap required by the common project model: proof-of-concept scope, milestones, milestone deliverables, dependencies, validation criteria and an indicative schedule. Each milestone should map to one or more research questions and measurable outcomes.

Assignment #2 — Automated PKI proof of concept and revocation experiments

- Build the private PKI designed in Assignment #1. Keep the Root CA private key offline after it signs the Intermediate CA. Configure OpenBao or HashiCorp Vault as the online Intermediate CA and document key storage, roles/policies, certificate profiles, AIA/CRL/OCSP URLs and trust-store installation.

- Enable ACME and use a standards-based ACME client to obtain and renew leaf certificates automatically. cert-manager is recommended when Kubernetes is used; Certbot, Caddy, lego or another well-documented ACME client is acceptable in a simpler environment. Demonstrate automatic renewal rather than manual re-issuance.
- Configure CRL publication and OCSP status service. If supported by the chosen PKI/server stack, also evaluate Delta CRLs and OCSP stapling. Must-Staple may be treated as an advanced/optional experiment if the selected issuer and TLS server can issue and enforce the required extension correctly.
- Simulate leaf-key compromise safely by copying a test certificate/private key to a controlled second TLS endpoint. At T0, revoke the original certificate (using ACME revokeCert or the PKI API/CLI) and continue probing the legitimate and simulated-compromised endpoints from instrumented clients.
- Measure the revocation timeline: T0, status storage at the CA, full/Delta CRL publication, OCSP response transition, stapled-response refresh, and the first time each relying party rejects the certificate. Report the resulting revocation-enforcement delay rather than only showing that a revocation command succeeded.
- Use at least two relying-party implementations, preferably one browser and one non-browser TLS client/runtime (for example curl/OpenSSL, a Java runtime or another service). Explicitly document whether each client performs CRL, OCSP, stapling or vendor-specific checks and do not infer behavior solely from RFC support.
- Compare the three strategies defined in Assignment #1 under the same threat model. In a compressed lab timescale, use a clearly longer-lived certificate for the CRL/OCSP scenarios and a much shorter lifetime for automatic-renewal experiments. Quantify exposure window, network/status traffic, renewal activity and operational dependencies.
- Inject controlled failures: make the CRL/OCSP service unavailable and observe soft-fail/hard-fail behavior; then make the Intermediate CA/ACME path unavailable around renewal time and observe how short-lived credentials fail. These experiments should distinguish security failure from availability failure.
- Answer the central question with evidence: under the selected use case, to what extent can sufficiently short-lived, automatically renewed certificates reduce or replace explicit revocation?
- Deliver reproducible configuration, scripts, certificate profiles, client commands, logs/PCAPs where relevant, timing data and a concise before/after comparison. The report should separate observations that are specific to the laboratory clients from conclusions that can reasonably generalize to public or enterprise PKI.

Group allocation Maximum two groups. Groups should study different revocation strategies or deployment contexts.

Further / research references

Larisch, J. et al. (2017). “CRLite: A Scalable System for Pushing All TLS Revocations to All Browsers.” IEEE Symposium on Security and Privacy. DOI: 10.1109/SP.2017.17 — Seminal CRLite paper; important for privacy-preserving push-based revocation.

Fooda, N. et al. (2026). “Comprehensive Revocation Checking at Scale: The Deployment of CRLite in Mozilla Firefox.” ACM SIGCOMM 2026, pp. 1739–1751. DOI: 10.1145/3789240.3829108 — Production-scale CRLite deployment and measurements.

Schanck, J. M. (2025). “Clubcards for the WebPKI: Smaller Certificate Revocation Tests in Theory and Practice.” IEEE Symposium on Security and Privacy, pp. 652–663. DOI: 10.1109/SP61157.2025.00128 — Modern compact data structure and empirical Web-PKI revocation dataset.

RFC 5280 — Internet X.509 Public Key Infrastructure Certificate and CRL Profile. DOI: 10.17487/RFC5280 — Normative CRL model.

RFC 6960 — X.509 Internet Public Key Infrastructure Online Certificate Status Protocol (OCSP). DOI: 10.17487/RFC6960 — Normative OCSP model.

RFC 7633 — X.509v3 TLS Feature Extension. DOI: 10.17487/RFC7633 — Basis for TLS Must-Staple.

RFC 8555 — Automatic Certificate Management Environment (ACME). DOI: 10.17487/RFC8555 — Automated certificate lifecycle and revocation request protocol.

RFC 8739 — Support for Short-Term, Automatically Renewed (STAR) Certificates in ACME. DOI: 10.17487/RFC8739 — Standards-based short-lived certificate lifecycle model.

5. Ransomware

Capacity	Maximum 2 groups

Ransomware as a networked intrusion lifecycle: initial access, credential abuse, lateral movement, data exfiltration, impact, resilience and safe experimental evaluation.

Context and challenge

Modern ransomware should not be treated as a single encryption program. It is usually the final impact stage of a broader intrusion lifecycle that can include phishing or vulnerability exploitation, credential theft, privilege escalation, remote-service abuse, lateral movement, discovery, exfiltration and deliberate targeting of backups. The contemporary ecosystem also includes ransomware-as-a-service (RaaS) and extortion models in which stolen data can be monetized even when encryption is prevented.

The ENISA Threat Landscape 2026 continues to identify ransomware as one of the most consequential short-term threats in Europe. For a network-security project, the most useful perspective is therefore not cryptanalysis of the payload but how the intrusion traverses identity and network boundaries: which initial-access paths are common, how credentials and remote-management protocols are abused, where segmentation fails, how command-and-control/exfiltration can be detected, and how containment limits blast radius.

Current defensive guidance emphasizes layered resilience. The CISA #StopRansomware Guide organizes measures around prevention, detection, response and recovery; NIST IR 8374 Rev. 1 (2026) maps ransomware outcomes to CSF 2.0 and explicitly includes governance, identity, backups, detection, response and recovery. The project should use these as reference frameworks while testing a small number of mechanisms empirically.

A useful technical model is to map a ransomware campaign to MITRE ATT&CK tactics and techniques, then identify observable events and enforcement points at each stage. Examples include anomalous authentication, remote-service use, internal scanning, unexpected SMB/RDP flows, unusual outbound transfers, high-rate file modification and attempts to impair recovery. The objective is not to reproduce a criminal payload but to understand which stages are visible to network and host defenses.

Backups are necessary but insufficient. A resilient design separates production credentials from backup administration, uses immutable/offline copies where appropriate, tests restoration, and assumes attackers may deliberately seek backup infrastructure. Likewise, segmentation must be validated experimentally: a diagram that shows VLANs or firewall zones is not evidence that compromised credentials cannot cross those boundaries.

Assignment #2 must therefore use a benign, reversible simulator operating only on synthetic data in isolated VMs. The scientific value comes from measuring whether selected controls detect or contain an attack sequence, how quickly they react, what false positives they create and how recovery behaves—not from implementing destructive malware.

Ransomware operations are increasingly identity-centric. Initial access may begin with phishing, stolen credentials, exploitation of an Internet-facing service or a trusted third party, after which attackers attempt to acquire more privileged identities and reach management, virtualization, backup and file services. This means network segmentation must be analysed together with identity boundaries and remote-administration paths. Mapping the scenario to MITRE ATT&CK helps students distinguish initial access, credential access, discovery, lateral movement, command-and-control, exfiltration, impact and recovery rather than treating encryption as the whole attack.

The extortion economy also changes the defensive objective. Double- and multi-extortion campaigns may steal data before encryption, threaten publication, or disrupt services even when encryption is blocked. Consequently, successful defence must include egress visibility, data-access monitoring and incident containment as well as endpoint anti-malware. The ENISA Threat Landscape 2026 provides contemporary European context, while NIST IR 8374 Rev. 1 translates ransomware risk into CSF 2.0 outcomes spanning governance through recovery.

Recovery is a security property that must be tested. Offline or immutable copies reduce the chance that a compromised administrative identity can destroy every restore point, but backups can still fail because of stale data, undocumented dependencies, unavailable keys or untested restoration procedures. Students should therefore define a recovery-time experiment on synthetic data and distinguish backup existence from recoverability. NIST SP 800-61 Rev. 3 is useful for placing containment, eradication and recovery in an incident-response lifecycle.

Evaluation should focus on observable evidence and control effectiveness. Examples include time from suspicious authentication to alert, time to isolate a compromised host, number of network segments reached, amount of synthetic data touched before containment, and restoration time. A benign simulator can replay a sequence of ATT&CK-like actions without encrypting or destroying real data. This yields a reproducible study of prevention, detection and resilience, consistent with the defensive playbooks in the CISA #StopRansomware Guide.

A useful research question is therefore not “can ransomware be stopped?” but “which controls break which stages of a realistic intrusion chain, and what residual risk remains?” Students should state their assumed attacker capabilities, identify the earliest reliable containment point, and explain what happens when that control fails. The resulting architecture should combine identity hardening, segmentation, monitoring, least privilege, protected recovery paths and tested incident response. This makes the project a network-security study of resilience under compromise rather than an exercise in malware construction.

Key concepts

- **Ransomware-as-a-Service (RaaS).** A service/ecosystem model in which operators provide malware and infrastructure to affiliates, changing attribution and increasing operational scale.
- **Double/multi-extortion.** Data theft and threats of disclosure or disruption supplement encryption, so preventing file encryption alone does not eliminate the incident.
- **Initial access.** Phishing, exposed remote services, credential theft and vulnerability exploitation are common starting points; the project should distinguish initial access from later impact.
- **Lateral movement.** Attackers use credentials, trust relationships and remote administration to reach additional systems; segmentation and identity controls determine blast radius.
- **Impact and recovery.** Encryption, service disruption and backup impairment are impact activities. Recovery quality depends on tested restore processes, clean credentials and trustworthy backups.
- **Detection engineering.** Network, identity and endpoint telemetry must be combined to recognize multi-stage activity; single indicators are often weak in isolation.
- **Containment.** Controls such as least privilege, segmentation, egress filtering and isolation aim to stop propagation/exfiltration before impact.
- **Benign emulation.** Coursework experiments must represent attack stages with harmless scripts/events and disposable synthetic files rather than real ransomware or self-propagating code.

Contextual reading

ENISA Threat Landscape 2026 — Current EU threat context; ransomware remains a high-impact threat.

CISA #StopRansomware Guide — Operational prevention and response guidance from CISA/NSA/FBI/MS-ISAC.

MITRE ATT&CK — Technique-oriented knowledge base useful for mapping the intrusion lifecycle and observable behaviors.

MITRE ATT&CK T1486 — Data Encrypted for Impact — Concrete ATT&CK technique for mapping the impact/encryption stage of a ransomware scenario to observable behaviors and mitigations.

Assignment #1 — State of the art

- Define a realistic ransomware threat model and intrusion path focused on network and identity security. Map the path to ATT&CK tactics/techniques and identify assumptions about attacker credentials, privileges and reachable systems.
- Review recent ransomware trends and explain RaaS, extortion, initial-access vectors, lateral movement, exfiltration and backup targeting. Use ENISA/NIST/CISA material together with peer-reviewed research or well-documented incident reports.

- Design a layered defensive architecture: identity/least privilege, segmentation, remote-service restrictions, egress monitoring, endpoint/network detection, protected backups and incident response. Explain what each control is expected to stop or reveal.
- Define measurable indicators for Assignment #2: detection time, containment time, number of reachable hosts, blocked flows, alert precision, amount of synthetic data affected and recovery time.
- Include an explicit safety design for the proof of concept: isolated VMs, generated test data, no persistence outside the lab, no self-propagation and a deterministic cleanup/reset method.
- Produce a milestone roadmap covering environment build, benign emulation, baseline measurement, control deployment, repeated experiments and final analysis.

Assignment #2 — Safe simulation

Safety constraint Real ransomware, destructive encryption, self-propagating malware and testing against third-party infrastructure are outside the scope of this coursework. Use benign simulations only.

Group allocation Maximum two groups; groups must choose different primary network entry/propagation scenarios.

- Build an isolated multi-host lab that represents at least an initial foothold, one internal service and a protected resource/backup role. Use only controlled credentials and synthetic data.
- Emulate a bounded multi-stage ransomware scenario using benign actions such as authentication attempts, remote administration, file-copy activity and reversible modification of generated files. Do not deploy real ransomware.
- Implement at least three controls from different layers (for example identity restriction, network segmentation/egress control, IDS/EDR-style alerting, immutable backup/recovery) and state the expected effect of each.
- Run baseline and defended trials with identical workloads. Measure detection/containment time, blast radius, false alerts and recovery time, and retain the relevant logs/PCAPs.
- Perform a recovery exercise from a known-clean state and explain which credentials/keys/configuration must also be trusted or rotated after compromise.
- Conclude by mapping the experimental evidence back to the NIST/CISA/ENISA guidance and identifying controls that were effective, ineffective or operationally expensive.

Further / research references

NIST IR 8374 Rev. 1 — Ransomware Risk Management — CSF 2.0 ransomware profile, June 2026.

NIST SP 800-61 Rev. 3 — Incident Response Recommendations and Considerations — Modern incident-response framework.

Kharraz, A., Robertson, W., Balzarotti, D., Bilge, L., & Kirda, E. (2015). Cutting the Gordian Knot: A Look Under the Hood of Ransomware Attacks. DIMVA 2015. DOI: 10.1007/978-3-319-20550-2_1.

Continella, A. et al. (2016). ShieldFS: A Self-healing, Ransomware-aware Filesystem. ACSAC 2016. DOI: 10.1145/2991079.2991110.

6. Intrusion Detection Systems (IDS)

Capacity	Maximum 2 groups

Network and host intrusion detection: signatures, anomaly/ML methods, encrypted traffic, datasets, evaluation quality, concept drift and adversarial robustness.

Context and challenge

Intrusion Detection Systems (IDS) observe events and generate evidence about activity that may violate a security policy. Classical systems such as Snort and Suricata use protocol parsing and rule/signature matching, while anomaly-based approaches learn or model normal behaviour and flag deviations. The distinction is important: signature systems are interpretable and effective for known patterns, whereas anomaly systems may identify novel behaviour but typically face higher false-positive rates and difficult model-maintenance problems.

The foundational taxonomy in NIST SP 800-94 remains useful for distinguishing network-, host- and behaviour-oriented detection, but modern deployments must deal with cloud workloads, containers, east-west traffic and pervasive encryption. TLS 1.3 and QUIC reduce payload visibility, shifting NIDS toward metadata, flow features, endpoint telemetry, TLS/QUIC fingerprints and cooperation with workload/runtime sensors. The 2024 survey on encrypted traffic classification provides context for what remains inferable without decrypting payloads.

Machine-learning IDS research is large but easy to evaluate badly. Recent surveys such as Hozouri et al. (2025) and Neto et al. (2025) highlight recurring problems: overuse of legacy datasets, class imbalance, data leakage, unrealistic train/test splits, poor transfer to production traffic, limited explainability and inadequate operational validation. High benchmark accuracy is therefore not equivalent to a deployable IDS.

Dataset choice is part of the threat model. CICIoT2023 contains 33 attacks executed in a 105-device IoT topology and offers both PCAP and extracted features, but a model that performs well there may not generalize to an enterprise or campus network. Students should explain which traffic population, attack classes and collection artefacts a dataset represents and should avoid mixing records from the same generated flow across training and test sets.

Adversarial robustness is now a central concern for ML-based NIDS. The 2026 survey Understanding the adversary: adversarial ML in NIDS identifies optimistic attacker assumptions, feature-space attacks and dataset dependence as persistent weaknesses. A scientifically strong project therefore evaluates not only detection rate but also false positives, robustness to drift/evasion, throughput/latency and the analyst effort created by alerts.

Assignment #2 should compare at least two detection paradigms under the same traffic and metrics—for example Suricata signatures versus a flow-based anomaly detector, or a conventional ML classifier versus a more interpretable baseline. The goal is to understand where each method succeeds and fails, not to maximize a single benchmark score.

An IDS is best understood as a measurement and decision pipeline: sensors collect packets, flows, logs or host events; parsers transform them into structured features; detection logic assigns meaning; and alerting/correlation turns detections into operational evidence. Placement matters because the same rule sees different information at an Internet edge, between application tiers, inside a Kubernetes node or at an endpoint. Students should therefore document what traffic the sensor can actually observe before evaluating algorithms. The architecture taxonomy in NIST SP 800-94 remains useful even though modern environments add cloud and container telemetry.

Evaluation must go beyond “accuracy.” Security operations care about precision, recall, false positives per unit time, detection latency, resource cost and the consequences of missed events. Class imbalance makes accuracy particularly misleading: if attacks are rare, a detector can achieve a high accuracy while being operationally useless. Recent IDS surveys such as Hozouri et al. (2025) emphasize these deployment and evaluation challenges, and students should present confusion matrices plus workload-aware metrics.

Dataset realism is itself a research problem. Public benchmarks are essential for repeatability, but many contain synthetic attacks, fixed lab topologies, obsolete protocols or artifacts that a model can exploit without learning the intended malicious behavior. A 2025 review of intrusion-detection datasets discusses collection processes

and feature design. Groups should justify dataset selection, separate training and test conditions correctly, and, where feasible, complement a public dataset with small traces generated in the course lab.

Pervasive encryption and adversarial adaptation define the modern frontier. A passive NIDS may classify TLS/QUIC flows using metadata while losing application semantics, as reviewed in the 2025 encrypted-traffic survey. ML-based NIDS can also be manipulated through evasion, poisoning or carefully crafted feature changes; the 2026 adversarial-ML NIDS survey provides a current taxonomy. A strong project should therefore discuss what an attacker knows about the detector and how robust conclusions remain under traffic drift or deliberate evasion.

The final system should also be evaluated as an operational sensor rather than only as a classifier. Students should specify alert semantics, severity and evidence, then inspect whether an analyst could distinguish a true incident from benign background activity. Detection latency and false alarms should be measured over a defined traffic interval, and resource overhead should be reported on the same host/network conditions. This requirement forces the project to connect model quality to the real purpose of an IDS: producing timely, actionable and trustworthy evidence.

Key concepts

- **Signature/rule-based detection.** Matches known protocol or content patterns; usually interpretable and efficient, but coverage depends on rule quality and timely updates.
- **Anomaly detection.** Models expected behaviour and flags deviations. It can expose previously unseen patterns but often suffers from false positives and unstable baselines.
- **NIDS vs HIDS.** NIDS sees network traffic and flow context; host/runtime IDS sees processes, files, system calls and identity. Modern detection frequently correlates both.
- **Encrypted-traffic metadata.** Even without decryption, endpoints, flow sizes, timing, TLS/QUIC handshake features and sequence behaviour may support classification, with privacy and robustness limitations.
- **Dataset validity.** Attack realism, traffic diversity, temporal split, class balance and feature-generation artefacts determine whether evaluation results are meaningful.
- **Concept drift.** Normal and malicious traffic evolve; detection thresholds/models trained on historical data can degrade over time.
- **Precision/recall and base rate.** In low-prevalence environments, even modest false-positive rates can overwhelm analysts; accuracy alone is misleading.
- **Adversarial robustness.** Attackers can change observable features, exploit blind spots or poison/evade learned models; ML-based IDS must be evaluated under plausible attacker knowledge and constraints.

Contextual reading

NIST SP 800-94 — Guide to Intrusion Detection and Prevention Systems — Foundational taxonomy and deployment guidance; dated in implementation details but still useful conceptually.

Snort 3 — Widely used open-source IDS/IPS and rule ecosystem.

Suricata documentation — Modern multi-threaded IDS/IPS/NSM with protocol-aware detection.

CICIoT2023 dataset — Modern real-IoT benchmark with PCAP and feature data.

Zeek Tutorial — Book of Zeek — Practical entry point for network-security monitoring and protocol/event telemetry complementary to signature IDS engines.

Assignment #1 — State of the art

- Define the deployment setting and threat model: where sensors sit, what traffic/host telemetry is visible, what encryption is present, and which attacks are in scope.
- Compare signature/rule-based, anomaly/statistical and ML/DL approaches. Explain the detection signal each consumes, interpretability, update requirements, compute cost and expected blind spots.
- Evaluate at least two candidate datasets or data-collection strategies. Discuss realism, age, attack coverage, class imbalance, temporal leakage and how the dataset influences the research question.
- Define a rigorous evaluation plan with confusion matrix, precision, recall, F1/PR-AUC where appropriate, false alerts per unit time, detection latency and throughput. Include confidence intervals or repeated runs.

- Address encrypted traffic and concept drift: state what features remain observable, what cannot be inferred reliably, and how the detector would be maintained as traffic changes.
- Produce a roadmap for Assignment #2 specifying baseline detector(s), dataset/traffic generator, milestones, deliverables, reproducibility artefacts and a robustness experiment.

Assignment #2 — Experimental work

Group allocation Maximum two groups; groups should investigate different detection approaches (e.g., signature/rule-based vs anomaly/ML, or network vs host telemetry).

- Deploy at least one production-grade IDS/NSM engine (e.g., Suricata or Snort) and one contrasting detector or configuration in an isolated lab.
- Generate or replay a labelled mix of benign and controlled attack traffic. Where public datasets are used, preserve a realistic train/test separation and document feature extraction.
- Measure detection quality and operational cost: precision/recall, false alerts, detection latency, CPU/memory/throughput, and analyst-readable evidence in the generated alerts.
- Perform at least one generalization or drift experiment, such as testing on a later time period, different benign workload, unseen attack variant or different network segment.
- For ML-based work, include a simple baseline and an adversarial/robustness check rather than reporting only the best model. Explain which attacker manipulations are realistic at packet/flow level.
- Deliver rules/models, configuration, scripts and data provenance sufficient to repeat the evaluation.

Further / research references

Hozouri, Mirzaei & Effatparvar (2025) — Recent, widely accessed IDS/ML/DL survey.

Neto et al. (2025) — Deep-learning IDS survey with focus on emerging technologies and deployment challenges.

Sharma & Lashkari (2024) — Encrypted-traffic identification/classification survey.

Espindola et al. (2026) — Adversarial machine learning in NIDS survey.

Pinto et al. (2025), A review on intrusion detection datasets: tools, processes, and features — Computer Networks 262, 111177; modern review of dataset construction and validity.

Mirsky, Y., Doitshman, T., Elovici, Y., & Shabtai, A. (2018). Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. NDSS 2018.

7. Intrusion Prevention Systems (IPS)

Capacity	Maximum 2 groups

Inline intrusion prevention: detection-to-enforcement pipelines, blocking semantics, false-positive risk, resilience, latency and evidence-based policy tuning.

Context and challenge

An Intrusion Prevention System extends detection into enforcement: traffic judged malicious can be dropped, rejected, rate-limited, quarantined or otherwise prevented from reaching the protected service. This makes an IPS qualitatively different from a passive IDS. A detection false positive in an IDS creates analyst workload; the same false positive in an inline IPS can create an outage. Prevention therefore introduces availability, latency and failure-semantics requirements that must be evaluated explicitly.

NIST SP 800-94 provides the classical IDPS framework, while current systems such as Suricata IPS mode and Snort 3 illustrate modern inline rule enforcement. Suricata can operate inline at layer 2 or layer 3 and applies drop/reject actions after protocol/stream inspection. Students should understand not only rule syntax but the packet-processing path, state tracking and what happens when inspection components fail or overload.

The main engineering problem is balancing prevention efficacy with service continuity. Inline inspection consumes CPU and memory and may become a bottleneck under load; encrypted traffic reduces payload visibility; asymmetric routing can break stateful inspection; and aggressive signatures can block legitimate applications. High availability and fail-open/fail-closed choices are therefore security decisions, not merely operations settings.

Modern prevention also extends beyond classic payload signatures. Reputation, behavioral thresholds, rate limiting, segmentation, identity-aware policy and kernel/runtime enforcement can complement network signatures. Machine-learning classifiers can propose suspicious flows, but automated blocking requires calibrated confidence and explainability because the cost of errors is higher than in passive alerting. Recent IDS surveys should be read critically from this prevention perspective rather than assuming every detector is suitable for inline action.

A good project compares detection-only and prevention modes under the same workloads. It should measure attack success rate, blocked legitimate traffic, latency/throughput overhead, time to enforce a new rule, and behavior under overload. Policy quality should be demonstrated with before/after traffic and service-level measurements rather than by showing that “the rule fired.”

Assignment #2 should be confined to the controlled SR lab. The objective is to design an evidence-based inline policy, demonstrate how attacks are blocked, and quantify the operational cost and false-positive risk.

IPS adds an enforcement decision to the detection pipeline. That change is fundamental: an IDS can tolerate an occasional false alarm, while an inline IPS that incorrectly blocks a legitimate authentication flow, database query or routing protocol can cause an outage. The project should therefore model prevention as a risk decision with explicit false-positive cost, not simply as “IDS plus blocking.” NIST SP 800-94 provides the classical distinction between detection and prevention and remains a useful baseline for deployment modes.

Inline placement creates state and availability questions. The enforcement point must parse traffic fast enough, maintain connection state, apply rules consistently and decide what to do when overloaded or unavailable. Fail-open preserves availability but can remove protection exactly during stress; fail-closed preserves policy but can turn an IPS failure into denial of service. Suricata’s IPS documentation gives a concrete implementation context for inline packet processing and stream-aware actions.

Encryption and distributed applications complicate prevention. A perimeter appliance may no longer see east-west service calls or application payloads, while host, service-mesh or eBPF enforcement can act closer to workloads but creates many policy points that must remain consistent. Students should compare where identity, flow metadata, decrypted endpoint context and network state are available, and distinguish a signature action such as “drop this exploit pattern” from higher-level authorization or segmentation policy.

Experimental evaluation should quantify both security and operational cost. Useful metrics include blocked malicious attempts, false blocks, time to enforcement, throughput and latency overhead, CPU/memory consumption, behavior under overload, and recovery after policy changes. The test plan should include benign traffic deliberately similar to the attack so that a rule cannot obtain an artificially perfect result simply by matching a unique lab artifact. This turns the project into an evaluation of safe inline policy rather than a demonstration that a single signature can drop a packet.

A central comparison should separate prevention quality from policy breadth. An IPS may block a narrow exploit signature very reliably while doing nothing about credential abuse, authorized-but-malicious API calls or lateral movement that looks syntactically normal. Conversely, broad anomaly-based blocking can create unacceptable availability risk. Students should therefore define which threat classes are suitable for automatic enforcement, which should remain alert-only, and where identity-aware authorization or segmentation is a better control than packet blocking. The conclusion should explicitly justify the chosen enforcement boundary.

Key concepts

- **Inline enforcement.** The IPS sits on or controls the forwarding path; a decision can directly drop or reject traffic rather than merely generating an alert.
- **Fail-open vs fail-closed.** On sensor failure or overload, traffic may bypass inspection or be blocked. The choice trades availability against security and must match the protected service.
- **Stateful protocol inspection.** TCP reassembly and application parsing reduce evasion opportunities but consume resources and are sensitive to asymmetric paths and ambiguous protocol behavior.
- **Rule action and confidence.** Alert, drop, reject and rate-limit semantics have different consequences; production prevention requires higher confidence than detection-only rules.
- **False-positive cost.** Prevention errors can interrupt legitimate service; evaluation must therefore include legitimate-traffic loss and service impact.
- **Throughput and latency.** An IPS must sustain target traffic rates without excessive queueing or packet loss; performance is part of the security evaluation.
- **Encrypted traffic.** TLS/QUIC constrain deep inspection unless termination/decryption is deliberately introduced; metadata- and endpoint-based controls become more important.
- **Defense in depth.** IPS is one control among segmentation, authentication, endpoint/runtime policy, WAF/API gateway and incident-response mechanisms.

Contextual reading

NIST SP 800-94 — Classical IDPS architecture, deployment and operational considerations.

Suricata — IPS Concept — Current explanation of inline traffic filtering and drop/reject semantics.

Snort 3 — Widely deployed open-source intrusion prevention platform.

Snort 3 Rule Writing Guide — Practical reference for constructing, testing and tuning detection/prevention rules and their actions.

Assignment #1 — State of the art

- Define the target service, expected traffic and attacker model. Draw the forwarding path and show exactly where the IPS sits and what it can observe.
- Explain the difference between IDS and IPS semantics, including false-positive consequences, fail-open/fail-closed behavior, stateful inspection and high-availability requirements.
- Compare at least two prevention approaches (e.g., signature-based inline IPS versus rate/behavior policy, or network IPS versus host/runtime enforcement) and justify which threats each can stop.
- Design a rule/policy lifecycle: detection evidence, test environment, staging, activation, rollback and logging. Explain how emergency rules differ from long-lived policy.
- Define performance and safety metrics: attack-block rate, legitimate-traffic block rate, throughput, latency, CPU/memory, rule-update convergence and behavior under overload.
- Produce the Assignment #2 roadmap with baseline, attack/replay workloads, policy milestones, measurement tooling and acceptance criteria.

Assignment #2 — Experimental work

Group allocation Maximum two groups; groups should study different prevention approaches or deployment architectures.

- Deploy an inline IPS in an isolated topology and establish a reproducible baseline with legitimate traffic before enabling blocking rules.
- Implement a small set of high-confidence policies that correspond to controlled attack scenarios. Preserve PCAPs/logs showing both detection evidence and enforcement outcome.
- Measure before/after attack success, legitimate request success, throughput and latency. Repeat trials and report variability.
- Introduce at least one intentionally over-broad rule or threshold in a safe test to demonstrate false-positive/service impact, then tune it and quantify the improvement.
- Evaluate one failure scenario such as sensor restart, CPU saturation, asymmetric path or rule-update error and document whether the design fails open or closed.
- Conclude with a deployment recommendation that states where automated blocking is justified and where alert-only operation is safer.

Further / research references

Hozouri et al. (2025) — Recent IDS/ML survey for detection methods and limitations.
Sharma & Lashkari (2024) — Encrypted traffic classification and visibility limits.
Zhao, Z. et al. (2020). Achieving 100Gbps Intrusion Prevention on a Single Server (Pigasus). OSDI 2020.
Wang, X. et al. (2019). Hyperscan: A Fast Multi-pattern Regex Matcher for Modern CPUs. NSDI 2019.

8. Honeypots & Honeynets for Detecting Intrusions

Capacity	Maximum 2 groups

Cyber deception with low-/medium-/high-interaction honeypots and honeynets: telemetry value, realism, containment, attacker detection and integration with defensive systems.

Context and challenge

Honeypots deliberately expose decoy services or systems so that interaction itself is suspicious and can be studied with high signal-to-noise ratio. A honeynet extends this idea to multiple coordinated decoys and monitoring components. Modern "cyber deception" broadens the concept further to include decoy credentials, services, data and network paths. The 2024 survey A comprehensive survey on cyber deception techniques to improve honeypot performance provides a useful taxonomy and emphasizes a central problem: once an attacker recognizes the deception, the value of the observation falls.

Interaction level is the core design choice. Low-interaction systems emulate a small protocol surface and are easier to contain, but expose fewer realistic behaviors. Medium/high-interaction systems allow richer sessions and can capture tools, commands and attack sequences, but create greater operational risk because the environment may be abused to attack third parties. Cowrie is a useful SSH/Telnet example because it supports both emulated and proxy/high-interaction modes and records sessions and transferred files.

T-Pot illustrates a modern multi-honeypot architecture: many deception services run together, with centralized logging/visualization and supporting tools such as Suricata. It is valuable for studying how diverse services generate different telemetry, but an "all-in-one" deployment should not replace a threat model. Students must decide what they want to observe, which attacker behaviors the decoy can realistically elicit, and how the data will feed detection engineering.

Containment is non-negotiable. A compromised high-interaction honeypot can become a pivot, proxy or malware-distribution node. Network egress must therefore be restricted, the environment must contain no sensitive data or credentials, and collection/retention must be designed with privacy and legal considerations in mind. The Cowrie proxy documentation and T-Pot warnings are useful starting points for this operational threat model.

The scientific challenge is to evaluate deception effectiveness rather than simply count attacks. Useful questions include: how quickly do different decoys attract interaction, what fraction of sessions proceed beyond automated scanning, which indicators can be exported into IDS/SIEM rules, how often do attackers detect the deception, and what telemetry is unique compared with passive network monitoring?

Assignment #2 should deploy decoys only inside authorized infrastructure, with strong egress containment. Students should compare at least two interaction/service profiles and transform observations into a small defensive artefact—e.g., Suricata/Sigma rules, IOC enrichment or an attacker-behavior taxonomy—then evaluate whether it improves detection on a separate test trace.

The central design variable in a honeypot is interaction level. Low-interaction systems emulate a narrow set of services and are easier to contain; high-interaction systems expose real operating systems or applications and can collect richer adversary behavior at substantially greater risk. Deception quality also matters: unrealistic banners, timing, file systems or network behavior can allow an attacker to fingerprint the trap. The 2024 cyber-deception survey frames deception as an adaptive engineering problem rather than merely "deploying a fake SSH server."

A honeynet adds network topology and multiple decoy roles, which makes containment more important. Outbound traffic from a compromised high-interaction honeypot must be restricted so the research infrastructure cannot become a platform for attacking others. Architectures such as T-Pot combine multiple honeypots and centralized logging, while Cowrie provides a focused SSH/Telnet emulation/proxy environment. Students should justify the fidelity they need and use the lowest-risk design that can answer the research question.

Telemetry is useful only if it can be transformed into defensible knowledge. Login attempts, commands, downloaded files, source addresses, protocol fingerprints and session sequences can help characterize behavior, but Internet-source data is biased by where the sensor is placed and can be polluted by scanners, bots or deliberate anti-analysis actions. The project should therefore define how events are normalized, deduplicated and clustered, and should avoid attributing an actor solely from an IP address or tool string.

A strong experimental loop is deception → observation → hypothesis → derived detector → independent validation. For example, a group may identify a recurring command sequence in Cowrie, convert it into a Suricata/Sigma-style detection hypothesis, and test that rule against separate benign and malicious lab traces. This explicitly connects deception to defensive improvement, which is the research purpose highlighted by The Honeynet Project, while keeping all active testing inside systems controlled by the students.

The project should also address reproducibility and research ethics. Publicly exposed honeypots can collect unsolicited interactions from real systems, so data minimization, retention, sanitization and legal/ethical constraints must be discussed even when payloads are malicious. A local or instructor-approved deployment is preferred for experiments that involve active manipulation. Reports should distinguish observations from attribution claims, preserve raw evidence where lawful, and document sensor configuration sufficiently for another group to reproduce the same deception and analysis pipeline.

Key concepts

- **Low/medium/high interaction.** Increasing realism exposes more attacker behaviour but also increases containment requirements and operational risk.
- **Deception fidelity.** A believable protocol stack, filesystem, timing and service banner make fingerprinting harder; unrealistic artefacts can reveal the honeypot.
- **Honeynet.** Multiple decoys plus capture/analysis infrastructure provide broader visibility and can model lateral movement between attractive targets.
- **Telemetry.** Session commands, authentication attempts, transferred files, protocol exchanges, PCAP and host events are the main research outputs.
- **Containment.** Egress filtering, isolation, disposable backends and resource limits prevent a compromised decoy from becoming an attack platform.
- **Detection engineering.** Honeypot observations become more valuable when converted into rules, IOCs or behavioural hypotheses and validated on independent traffic.
- **Attacker awareness.** Sophisticated adversaries may fingerprint virtualization, emulation errors, unrealistic content or network behavior; deception is therefore an adversarial design problem.
- **Ethics/privacy.** Collected sessions may include credentials, personal data or malicious payloads; retention, access and publication require sanitization and clear scope.

Contextual reading

The Honeynet Project — Long-running community resource on honeypots, research and operational practices.

T-Pot — Actively maintained multi-honeypot platform with centralized telemetry.

Cowrie — Widely used SSH/Telnet medium/high-interaction honeypot documentation.

OpenCanary — Documentation — Lightweight low-interaction decoy platform useful as a contrast to Cowrie and multi-honeypot T-Pot deployments.

Assignment #1 — State of the art

- Define the intended adversary and observation goal. Choose the services and interaction level based on the behaviour you want to elicit, not merely tool popularity.
- Compare low-, medium- and high-interaction honeypots in realism, telemetry richness, detectability, maintenance and containment risk. Include at least two current projects.
- Design the honeynet architecture: decoy placement, management network, logging pipeline, time synchronization, PCAP/host telemetry and egress restrictions.
- Explain likely honeypot fingerprinting techniques and how the design reduces obvious artefacts without attempting to make unbounded claims of “undetectability.”

- Define data-handling policy: what is captured, how credentials/payloads are quarantined, retention, sanitization and what can be included in the report.
- Create the Assignment #2 roadmap with deployment, controlled interaction generation, observation period, telemetry analysis, rule/IOC derivation and validation milestones.

Assignment #2 — Practical work

Safety constraint Default deployment is laboratory-only. Internet exposure requires explicit instructor approval and a documented containment/monitoring plan.

Group allocation Maximum two groups; use different honeypot/honeynet platforms or clearly different deception goals.

- Deploy at least two different decoy profiles (e.g., SSH/Telnet and another service, or different interaction levels) in an isolated/authorized environment with strict outbound containment.
- Generate controlled benign interactions so that the logging pipeline and analysis can be validated before any Internet-facing or broader experiment is considered.
- Collect and characterize sessions using duration, commands, protocol requests, credentials used, file-transfer attempts and source behavior. Do not publish live credentials or unvetted payloads.
- Compare telemetry richness and resource/maintenance cost between the selected honeypots and identify evidence of automated scanners versus deeper interactive behavior.
- Derive a small set of detection artefacts (e.g., Suricata/Sigma rules or IOC/behavior patterns) and validate them on a separate controlled trace to estimate usefulness and false positives.
- Evaluate containment by demonstrating that a compromised/interactive decoy cannot reach protected lab assets or unrestricted external targets.

Further / research references

Javadpour et al. (2024), A comprehensive survey on cyber deception techniques to improve honeypot performance — Computers & Security 140, 103792.

Provos, N. (2004). A Virtual Honeypot Framework. 13th USENIX Security Symposium.

Pa, Y. M. P. et al. (2015). IoTPOT: Analysing the Rise of IoT Compromises. WOOT 2015.

9. Increasing Network Security with eBPF

Capacity	Maximum 2 groups

eBPF/XDP/LSM for high-performance network visibility and runtime enforcement, with attention to verifier safety, privilege boundaries and the new attack surface created by programmable kernel hooks.

Context and challenge

eBPF allows verified programs to run at selected Linux kernel hook points and exchange state with user space through maps and ring buffers. For network security this creates a continuum from very early packet processing with XDP, through traffic-control and socket/cgroup hooks, to runtime security through tracing and BPF LSM. The Linux BPF documentation and Cilium BPF/XDP reference are useful technical foundations because they explain the execution model rather than treating eBPF as a product feature.

The verifier is central to the security model. It statically reasons about program control flow, memory access, pointer types and bounded execution before bytecode is allowed to run in kernel context. The eBPF verifier documentation makes clear that this safety model is not equivalent to "untrusted code may run freely": program loading remains privileged and capabilities/program types matter. Students should understand both verifier guarantees and the operational privilege boundary.

For packet processing, XDP executes near the driver receive path and can drop or redirect traffic before much of the normal network stack runs, making it attractive for DDoS filtering and high-rate telemetry. tc/cgroup/socket hooks provide richer context later in the stack. For host/runtime security, BPF LSM can enforce mandatory-access-control decisions, while platforms such as Tetragon combine eBPF sensors with process/network events and workload identities.

eBPF is also part of the attack surface. The USENIX Security 2023 paper Cross Container Attacks: The Bewildered eBPF on Clouds demonstrates that powerful tracing capabilities and over-privileged container environments can undermine isolation even without a kernel exploit. A rigorous project must therefore study who may load BPF, what hooks are exposed, how programs are audited/signed, and how capabilities are constrained—not only how defenders can use eBPF.

Research systems such as bpfbox and BPFContain show how eBPF can implement fine-grained confinement and container-aware policy. Their value for this project is conceptual: they expose the design trade-off between semantic, high-level policy and low-level kernel hooks, and provide a basis for measuring enforcement overhead and policy coverage.

Assignment #2 should build a small, auditable eBPF security component rather than a large framework. Suitable goals include XDP-based filtering with counters, connection telemetry correlated with processes, or a BPF-LSM policy protecting a specific resource. Students should measure correctness, overhead and failure behavior, and should separately assess the privileges introduced by their own eBPF deployment.

eBPF is not a single security product but an execution and instrumentation substrate in the Linux kernel. Programs are loaded into defined hook types, verified for safety properties, optionally JIT-compiled, and communicate state through maps or ring buffers. The verifier is therefore part of the trusted computing base: it constrains program behavior before kernel execution, while kernel privileges and loader policy determine who may attach programs. The Linux BPF documentation and verifier documentation should be used to understand these mechanics before comparing tools.

Different hooks provide different security semantics. XDP can act before most of the networking stack and is suitable for very early filtering; tc sees packets later with richer networking context; cgroup and socket hooks can associate flows with workloads/processes; tracepoints/kprobes observe kernel/runtime events; and BPF LSM supports policy decisions at Linux Security Module hooks. A good design selects a hook based on the question rather than assuming "eBPF" automatically means faster firewalling.

Production platforms illustrate the split between data-plane policy and runtime observability. Cilium's BPF/XDP reference shows how eBPF implements networking, load-balancing and policy in container environments, whereas Tetragon focuses on process/kernel observability and enforcement. Students should compare what

state each system observes, how identity is represented, and which enforcement actions occur before or after an event reaches the application.

The technology also creates a security boundary of its own. eBPF historically required elevated privileges, verifier and JIT bugs can become kernel attack surfaces, and cloud/container environments must prevent untrusted workloads from obtaining unsafe BPF capabilities. The USENIX Security paper Cross Container Attacks: The Bewildered eBPF on Clouds is useful for understanding this dual use. Assignment #2 should measure both benefit (visibility, blocking latency, throughput) and cost (CPU, memory, event loss, privileges and operational complexity).

Portability is another practical research dimension. eBPF programs often depend on kernel data structures and hook availability, while BTF and CO-RE mechanisms aim to reduce kernel-version coupling. A prototype should record kernel, compiler/libbpf and tool versions, explain whether it runs in native/kernel or userspace fallback mode, and quantify event loss or enforcement gaps under load. This makes it possible to distinguish the elegant conceptual model of "programmable kernel security" from the real engineering constraints of deploying the same policy across heterogeneous Linux systems.

Key concepts

- **Verifier.** Static analysis executed when a BPF program is loaded; it constrains control flow and memory/pointer use so accepted programs can execute safely in kernel context.
- **JIT and hook points.** Verified bytecode may be JIT-compiled and attached to XDP, tc, cgroup/socket, tracepoint/kprobe or LSM hooks; each hook exposes different context and performance trade-offs.
- **XDP.** Very early receive-path hook suitable for high-rate filtering, DDoS mitigation and lightweight telemetry before the normal network stack.
- **Maps and ring buffers.** Shared data structures allow kernel programs to maintain state and communicate observations to user-space control/analytics components.
- **BPF CO-RE/BTF.** BTF type information and Compile Once – Run Everywhere reduce kernel-version coupling and improve portability of modern eBPF tooling.
- **BPF LSM.** eBPF programs attached to Linux Security Module hooks can implement audit or mandatory access-control decisions at runtime.
- **Privilege/attack surface.** Loading powerful BPF programs is itself security-sensitive; capabilities, container privileges and kernel vulnerabilities must be part of the threat model.
- **Observability vs enforcement.** Telemetry can be safer to deploy than blocking; enforcement must additionally consider false positives, semantic coverage and availability impact.

Contextual reading

Linux kernel BPF documentation — Primary documentation for program types, maps, verifier, BTF and kernel APIs.

eBPF verifier documentation — Clear explanation of load-time safety verification.

Linux BPF LSM programs — Kernel documentation for attaching BPF to LSM security hooks.

Cilium BPF and XDP Reference Guide — Deep technical reference for networking hooks and BPF architecture.

Tetragon — Current eBPF runtime security observability and enforcement project.

libbpf-bootstrap — Small, current reference applications demonstrating libbpf, CO-RE, maps and ring buffers for practical eBPF development.

Assignment #1 — State of the art

- Explain the eBPF execution and safety model: bytecode/JIT, verifier, maps, helpers, privileges and lifecycle. Distinguish verifier safety from authorization to load a program.
- Compare relevant hook points (XDP, tc, cgroup/socket and BPF LSM) for at least two security use cases, discussing information available, enforcement capability and expected overhead.
- Study one production project (e.g., Cilium/Tetragon) and one research mechanism (e.g., bpfbox/BPFContain) to connect practical design with peer-reviewed security work.
- Include a threat model for eBPF itself: malicious or compromised loader, over-privileged container, unsafe helper/program type, kernel attack surface and policy bypass.

- Define measurable criteria for the prototype: packet/event rate, CPU overhead, latency, drop/enforcement correctness, memory/map use and false-positive behavior where applicable.
- Produce a milestone roadmap covering minimal program, user-space controller/telemetry, policy tests, benchmark harness, threat-model validation and reproducibility artefacts.

Assignment #2 — Prototype and evaluation

Group allocation Maximum two groups; groups must select different eBPF security applications or hook/architecture choices.

- Implement a focused eBPF network/runtime security prototype in an isolated Linux environment, using libbpf/CO-RE or another justified toolchain.
- Demonstrate a clear security property, such as blocking a defined network class at XDP, restricting a specific resource through BPF LSM, or correlating connections to process identities.
- Validate correctness with positive and negative test cases and compare against a conventional mechanism where meaningful (e.g., nftables for packet filtering).
- Measure overhead and scalability across repeated runs. Report CPU, throughput/latency/event rate and resource use, not just functional success.
- Audit the privileges required to load/run the program and demonstrate what changes when those privileges are removed or constrained. Discuss exposure in containerized environments.
- Package source, build instructions, kernel requirements and test scripts; document verifier failures or portability constraints encountered during development.

Further / research references

Findlay, Somayaji & Barrera, bpfbox — ACM CCSW 2020 process-confinement paper.

Findlay, Barrera & Somayaji, BPFContain — Container security with eBPF.

He et al., Cross Container Attacks: The Bewildered eBPF on Clouds — USENIX Security 2023 analysis of eBPF-related container attack surface.

10. The European Digital Identity Wallet Architecture and Reference Framework (ARF)

Capacity	Maximum 2 groups

EUDI Wallet architecture, trust model and interoperability: issuance, presentation, selective disclosure, device-bound keys, relying-party trust and conformance under the current ARF.

Context and challenge

The European Digital Identity Wallet (EUDI Wallet) is a regulated, interoperable digital-identity ecosystem rather than a single application. Regulation (EU) 2024/1183 amended eIDAS and establishes the legal basis; the current technical baseline is the EUDI Wallet Architecture and Reference Framework (ARF). As of 2026 the latest ARF release is v3.0.0, aligned with additional implementing regulations and including a functional conformance assessment framework. Students must use the current tagged ARF rather than older presentations of "eIDAS 2.0."

The architecture separates issuers, wallet providers/instances, holders, relying parties, trust services, registries and supervisory/conformance functions. Security analysis must therefore follow trust relationships and protocol boundaries: who attests the wallet, how issuer keys are trusted, how a relying party requests data, how the holder consents, how a verifier validates provenance and freshness, and what happens when credentials, wallet instances or relying-party registrations are revoked or updated.

Remote issuance and presentation increasingly rely on the OpenID Digital Credentials protocols. OpenID4VCI 1.0 standardizes credential issuance and OpenID4VP 1.0 standardizes presentation. The OpenID4VC High Assurance Interoperability Profile (HAIP) 1.0 narrows choices for high-assurance interoperability. These specifications should be read together with the ARF because implementation security depends on protocol profiles, key binding and trust configuration, not just credential format.

Data minimization is central. Selective-disclosure technologies allow a holder to reveal only the attributes needed for a transaction instead of an entire credential. RFC 9901 (SD-JWT) standardizes selective disclosure for JWT claims, while the IETF SD-JWT VC draft defines a credential profile. ISO mdoc is also relevant for mobile driving licences and proximity use cases. Students should distinguish selective disclosure from anonymity: a credential can disclose fewer claims while still being linkable through other protocol or ecosystem identifiers.

Wallet security also depends on device-bound keys, secure key storage, wallet-instance attestation, issuer/verifier authentication, transaction binding and defenses against malicious relying parties. Privacy analysis must consider over-requesting, correlation, identifier reuse, logging and ecosystem governance in addition to cryptography. A good threat model therefore includes compromised holder devices, malicious/compromised issuers, dishonest relying parties, compromised backend services and failures in trust lists/registries.

The practical goal should be an interoperable miniature transaction, not a complete wallet product. The EU reference implementation and conformance resources can provide components and test vectors. Students should implement or integrate issuance and presentation for a small use case, demonstrate selective disclosure or device/key binding, and evaluate the trust and privacy properties with protocol traces and negative tests.

The EUDI Wallet is not simply a mobile identity application; it is an ecosystem in which legal roles, trust lists, issuers, wallet providers, relying parties and credential formats must interoperate across Member States. The current Architecture and Reference Framework v3.0.0 (July 2026) should be treated as the architectural baseline, while Regulation (EU) 2024/1183 and implementing acts define legal requirements. Students should separate legal obligations, ARF architectural choices and protocol specifications rather than treating them as one document.

Credential issuance and presentation are distinct protocol phases. OpenID4VCI 1.0 standardizes how a wallet obtains credentials from an issuer, while OpenID4VP 1.0 standardizes how a verifier requests and receives presentations. This separation allows the project to trace authentication, authorization, nonce/state binding, proof of possession, credential selection and verifier policy across an end-to-end flow. It also exposes where phishing, replay, malicious verifier behavior or issuer compromise must be considered.

Privacy depends on more than encrypted transport. Selective disclosure formats such as SD-JWT (RFC 9901) can reveal only selected claims, but unlinkability also depends on identifiers, keys, metadata and presentation patterns. High-assurance ecosystems additionally need holder/device binding, issuer/wallet attestation and controlled relying-party trust. HAIP 1.0 provides a useful interoperability/security profile for understanding how multiple specifications are constrained into a deployable high-assurance subset.

Conformance is a security mechanism because subtle protocol deviations can create interoperability failures or exploitable inconsistencies. The OpenID Foundation now publishes OpenID4VP/OpenID4VCI conformance suites and self-certification, which provides a concrete way for students to distinguish "our prototype seems to work" from "the implementation conforms to a profile." A good Assignment #2 should implement a limited issuer-wallet-verifier scenario, capture protocol messages, exercise selective disclosure and at least one failure case, and map the observed behavior back to ARF roles and trust assumptions.

The project should finally distinguish privacy-preserving disclosure from governance of who is allowed to ask for data. A cryptographic format can support selective disclosure, yet a relying party may still request more attributes than necessary or correlate presentations through metadata. Students should therefore model purpose limitation, verifier authorization/registration, user consent and transaction logging alongside cryptographic proofs. The strongest result will show how protocol messages, trust anchors and policy decisions jointly produce—or fail to produce—the privacy and assurance properties claimed by the wallet ecosystem.

Key concepts

- **Issuer–Holder–Verifier model.** Issuers create credentials, holders store/control them in wallets, and verifiers/relying parties request and validate presentations under an explicit trust model.
- **PID and (Q)EAA.** Person Identification Data and electronic attestations of attributes represent different credential roles and assurance/governance requirements in the EUDI ecosystem.
- **OpenID4VCI / OpenID4VP.** OAuth-based protocols for remote issuance and presentation; security depends on endpoint authentication, authorization, transaction binding and supported credential formats.
- **Selective disclosure.** The holder reveals only necessary claims; cryptographic disclosure minimization does not by itself prevent correlation across transactions.
- **Device/key binding.** Credentials or presentations can be bound to key material held by the legitimate wallet instance, reducing replay/theft risk.
- **Wallet and issuer attestation.** Attestations communicate assurance about wallet instances/providers or issuer capabilities and become part of the relying party trust decision.
- **Trust lists/registries and relying-party registration.** Interoperability requires common trust anchors and mechanisms to determine who may issue/request particular data.
- **Conformance.** A wallet ecosystem needs testable normative requirements; ARF v3 introduces a functional conformance assessment framework and shared test cases.

Contextual reading

EUDI Wallet ARF v3.0.0 — Current architecture, roles, trust model, normative high-level requirements and technical specifications.

Regulation (EU) 2024/1183 — Legal amendment establishing the European Digital Identity Framework.

EUDI Wallet Reference Implementation — Open-source implementation resources and components.

OpenID4VP/OpenID4VCI conformance tests and self-certification — Current conformance infrastructure for final OpenID4VC profiles.

EUDI Wallet — Open-source repositories — Official source repositories for wallet reference components, useful for moving from ARF requirements to implementable components.

Assignment #1 — State of the art

- Use the current ARF release to map ecosystem actors, trust relationships and the lifecycle of at least one credential from issuance to presentation and revocation/update.
- Explain the role of OpenID4VCI, OpenID4VP and applicable credential formats/profiles (e.g., SD-JWT VC or mdoc). Identify which choices are mandated/profiled by the ARF/technical specifications.

- Develop a threat model covering holder device compromise, malicious relying party, issuer compromise, replay, correlation/privacy leakage, trust-list/registry manipulation and backend compromise.
- Analyse data minimization and unlinkability separately. Show which mechanisms support selective disclosure and which metadata/identifiers could still enable correlation.
- Define a small use case and the assurance/security properties to be tested in Assignment #2, including negative cases and conformance/interoperability criteria.
- Produce an implementation roadmap with selected reference components/libraries, milestones for issuance, presentation, verification, negative tests and protocol-trace analysis.

Assignment #2 — Pilot/prototype

Group allocation Maximum two groups; groups should select different EUDI use cases or technical focus areas.

- Implement or integrate a minimal issuer–wallet–verifier flow for a controlled use case using EUDI-compatible reference components or standards-conformant libraries.
- Demonstrate credential issuance and presentation, including at least one privacy/security mechanism such as selective disclosure, holder/key binding or verifier authorization.
- Capture and annotate protocol messages without exposing real personal data. Explain which fields are security-critical and which identifiers/metadata can contribute to linkability.
- Implement negative tests: invalid/expired credential, wrong audience/verifier, replayed presentation, altered disclosure or untrusted issuer, as applicable to the chosen profile.
- Evaluate the implementation against relevant ARF/conformance requirements and document interoperability limitations or ambiguities found.
- Conclude with a trust/privacy analysis that distinguishes cryptographic guarantees from governance, registration and operational assumptions.

Further / research references

OpenID4VCI 1.0 — Final issuance specification.

OpenID4VP 1.0 — Final presentation specification.

OpenID4VC HAIP 1.0 — High-assurance profile.

RFC 9901 — SD-JWT — Selective disclosure standard.

SD-JWT VC IETF draft — Current credential profile under standardization.

Ansaroudi, Z. E. et al. (2025). Navigating secure storage requirements for EUDI Wallets: a review paper. EURASIP Journal on Information Security.

Privacy evaluation of the European Digital Identity Wallet's Architecture and Reference Framework (2026). Computers & Security, 160, 104707.

11. Laboratory: MITM with Ettercap, Credential Exposure and Metasploit Exploitation

Capacity	Maximum 2 groups

ARP-based MITM, protocol exposure, credential abuse, vulnerability exploitation, and experimental evaluation of layered defenses.

Context and challenge

The project studies an end-to-end attack path in a local network: establish an ARP-based man-in-the-middle position with Ettercap, determine what application information becomes observable or modifiable, extend the experiment to credential reuse and vulnerability exploitation with Metasploit, and then repeat the same tests after defensive controls are deployed. The emphasis is on observable security properties and measurable changes between the baseline and defended configurations.

In an IPv4 Ethernet broadcast domain, ARP has no built-in authentication. Forged ARP replies can therefore alter the victim and gateway ARP caches so that traffic is forwarded through the attacker. Students should analyse the ARP exchanges, the resulting IP-to-MAC mappings, the forwarding path and the packet evidence that confirms successful on-path positioning. This mechanism corresponds to ARP cache poisoning (MITRE ATT&CK T1557.002).

Once the attacker is on path, the next step is to compare what can be learned from different application protocols. Plaintext traffic may reveal credentials, commands and content and may also permit modification in transit. With correctly validated TLS 1.3 (RFC 8446), the attacker can still relay packets but should not be able to recover or forge protected application data. HSTS (RFC 6797) should be introduced as the browser mechanism that instructs a known origin to use HTTPS and prevents ordinary HTTP downgrade attempts.

An optional mitmproxy experiment can then examine the role of trust anchors. A laboratory client that explicitly trusts a local interception CA will accept a certificate generated by the proxy, allowing the group to inspect the resulting TLS sessions and compare the certificate chain and traffic visibility with the normal TLS case. The mitmproxy certificate documentation provides the practical setup for this experiment.

The Metasploit phase extends the experiment from traffic observation to attack impact. First, credentials recovered from an intentionally plaintext service can be tested against the corresponding service using a suitable Metasploit auxiliary/login module or the native protocol client. Second, the group should enumerate one service on Metasploitable 2 or Metasploitable 3, identify a documented vulnerability, select the corresponding Metasploit exploit module and obtain a shell or Meterpreter session. These two activities expose different failure modes: compromise of authentication material and exploitation of a vulnerable network service.

The credential-reuse experiment should record what authentication material was visible, whether it is sufficient for a new login, and how additional controls change the result. Relevant controls include strong authentication, multifactor authentication where applicable, least privilege, reduced credential/session lifetime and secure handling of authentication tokens. The outcome should distinguish loss of confidentiality from the later ability to authenticate successfully.

Before selecting an exploit, use Nmap service and version detection (or equivalent evidence) to identify the exposed service and version. The vulnerability-exploitation experiment should then record the discovered service and version, the associated vulnerability or CVE where applicable, the Metasploit module and payload used, and the resulting session. The same exploit should then be repeated after the service is patched, disabled, filtered or otherwise hardened. This creates a direct before/after measurement of exploitability under a controlled configuration change.

The scientific component is a layered evaluation of the complete attack chain. Students should model at least five stages: (1) ARP-based on-path positioning; (2) observation or modification of application traffic; (3) reuse of exposed credentials or session material; (4) exploitation of a vulnerable service; and (5) access to additional protected resources after compromise. For each stage, the project should identify the required attacker

capability, the observable evidence of success, and the control points that can prevent, protect, contain or detect the activity.

Defense should be evaluated across several layers. At Layer 2, Dynamic ARP Inspection (DAI) with DHCP snooping can validate ARP mappings and block forged bindings; static ARP entries may be examined only as a small-lab comparison. Network segmentation and access-control rules can reduce the systems reachable from the victim or from a compromised host. At the transport/application layer, TLS 1.3 with correct certificate validation protects confidentiality and integrity even when the attacker remains on path, while HSTS reduces downgrade opportunities and mTLS can add client authentication where appropriate. At the identity layer, multifactor authentication, least privilege and short-lived credentials can reduce the usefulness of captured authentication material. At the host/service layer, patching, service minimization and host firewalls remove or constrain exploit paths. IDS/IPS, ARP monitoring, switch logs, authentication logs and endpoint/service telemetry provide detection and evidence across the chain.

The evaluation should compare the same workload before and after selected defenses are enabled. Useful measurements include ARP-poisoning success rate and establishment time; changes in ARP mappings and observed traffic path; plaintext information exposed; TLS-protected information remaining confidential; credential-reuse success; exploit/session success rate; detection latency and false alerts; service availability; and whether segmentation blocks access to a downstream protected host. Measurements should be repeated where variability is relevant rather than reported from a single run.

PCAPs inspected with Wireshark, ARP tables, TLS certificate chains, Metasploit output, authentication events, switch/IDS alerts and service logs should be correlated into a common timeline. The final analysis should show where the attack chain succeeds, where it stops, what evidence is produced, and what operational cost or side effect each defense introduces. This allows direct comparison between preventive, protective, containment and detection mechanisms.

Key concepts

- **Adversary-in-the-middle / on-path position.** A network position from which traffic can be observed, relayed or modified between communicating endpoints.
- **ARP cache poisoning.** Manipulation of IPv4-to-MAC mappings in a local broadcast domain so traffic is redirected through an attacker-controlled host.
- **End-to-end channel protection.** TLS protects application confidentiality and integrity across an untrusted forwarding path when endpoint authentication and certificate validation remain sound.
- **Trust anchor and TLS interception.** A client accepts certificates that chain to trusted roots; adding a laboratory CA changes the set of issuers the client accepts and makes explicit interception possible.
- **HSTS.** Browser policy that requires HTTPS for a known origin and blocks ordinary attempts to fall back to HTTP.
- **Credential reuse and vulnerability exploitation.** Reuse of exposed authentication material and exploitation of vulnerable software are distinct attack paths with different controls and evidence.
- **Metasploit module, payload and session.** A module implements the selected exploit or auxiliary action; the payload defines the action after successful exploitation; the resulting shell or Meterpreter session provides observable evidence of compromise.
- **Prevention, protection, containment and detection.** Controls may stop an attack stage, preserve a security property despite it, limit the attacker's reach, or generate evidence and alerts.
- **Evidence correlation.** Packet captures, ARP state, certificate chains, Metasploit output and network/host logs are combined to reconstruct the attack path and evaluate defenses.

Contextual reading

Rapid7 — Metasploit Basics — Introductory overview of Metasploit concepts and workflow before using the more specific exploitation tutorials.

Nmap — Service and Version Detection — Official guide to identifying the actual service and software version exposed on a port before selecting a vulnerability or Metasploit module.

Wireshark User's Guide — Practical reference for packet capture and protocol analysis; use it to inspect ARP exchanges, plaintext traffic, TCP/TLS handshakes and PCAP evidence.

RFC 826 — Address Resolution Protocol (ARP) — Primary reference for how IPv4 addresses are resolved to Ethernet/MAC addresses; useful before studying ARP cache poisoning.

Ettercap project — Current source/documentation for controlled LAN on-path and ARP-poisoning demonstrations.

MITRE ATT&CK T1557.002 — ARP Cache Poisoning — Threat-model framing, procedure examples, mitigations and current detection guidance for ARP-based adversary-in-the-middle activity.

mitmproxy — How mitmproxy works — Explains TLS interception, certificate generation and why the client trust store determines whether interception succeeds; pair with the certificate setup guide.

OWASP Transport Layer Security Cheat Sheet — Practical reference for TLS configuration, certificate validation and mutual TLS; see also the HSTS Cheat Sheet for browser HTTPS enforcement.

Cisco — Dynamic ARP Inspection — Concrete reference for DAI, DHCP-snooping bindings and invalid-ARP filtering; useful even when the laboratory switch uses an equivalent mechanism.

Rapid7 Metasploit Framework — Manual Exploitation — Official tutorial for searching, selecting, configuring and running one exploit module against an authorized target.

Rapid7 — Setting Up a Vulnerable Target — Official guidance for building a contained intentionally vulnerable target, including Metasploitable 3.

Rapid7 — Metasploitable 2 — Official setup guide for the intentionally vulnerable VM used as a contained target.

Rapid7 — Metasploitable 2 Exploitability Guide — Service-by-service examples suitable for selecting and justifying a bounded exploit-validation exercise.

Rapid7 — Working with Payloads — Explains payload selection and the distinction between command-shell and Meterpreter-style payloads.

Rapid7 — Managing Meterpreter and Shell Sessions — Official reference for handling and terminating laboratory sessions.

Rapid7 — Metasploit Modules — Explains exploit, auxiliary and post modules and how to search by platform, application or CVE before selecting a laboratory module.

Assignment #1 - State of the art and laboratory design

- Design the laboratory topology with an attacker, client/victim, service target and a separate protected resource where useful. Define the complete attack path: ARP-based positioning, traffic exposure, credential reuse, vulnerability exploitation and post-compromise reach.
- Explain ARP operation and ARP cache poisoning at packet level and relate the positioning stage to MITRE ATT&CK T1557.002. Define the evidence used to determine success: ARP-cache changes, MAC/IP mappings, packet path, forwarding state and PCAP observations.
- Compare at least one plaintext application flow with HTTPS/TLS 1.3. Explain what is visible or modifiable in each case, introduce HSTS as HTTPS-enforcement policy, and analyse certificate validation and trust anchors. The optional mitmproxy experiment should show how adding a laboratory CA changes the observed certificate chain and traffic visibility.
- Plan two Metasploit activities: (A) reuse of credentials recovered from the plaintext experiment against the corresponding service, and (B) exploitation of one documented vulnerability on Metasploitable 2 or 3. For the exploitation activity identify the service/version, vulnerability or CVE where applicable, Metasploit module, payload/session type and expected observable result.
- Build a defense matrix covering at least four layers or control categories. Suitable controls include DAI/DHCP snooping or an equivalent Layer-2 mechanism; TLS/HSTS/mTLS; multifactor authentication and least privilege; patching, service hardening and host firewalling; network segmentation; and ARP/IDS/authentication/endpoint monitoring. For each control define the attack stage affected, expected security effect and measurable success criterion.
- Produce the Assignment #2 roadmap with baseline measurements, attack execution, defense deployment, repeated trials and final comparison. Define the metrics, logs and packet evidence to be collected for each stage.

Assignment #2 - Experimental attack chain and defense evaluation

Mandatory authorization boundary All MITM and Metasploit activity must remain inside laboratory VMs and networks owned/controlled by the group. Do not target campus infrastructure, public services, other students' hosts or any third-party systems.

Group allocation Maximum two groups. Groups must use different primary service/protocol scenarios or different defence/evaluation emphases.

- Establish the laboratory baseline: record ARP tables, routes, service versions, certificate chains, authentication behavior and relevant network/host logs. Use Nmap service/version detection where needed to identify exposed services and keep the resulting output as part of the baseline evidence.
- Use Ettercap to establish ARP-based MITM positioning. Preserve PCAP and ARP-table evidence of the path change, then compare a plaintext flow with a TLS-protected flow. Use Wireshark to inspect and annotate the relevant ARP, application and TLS packets.
- Optionally repeat the TLS experiment with an explicitly trusted laboratory CA and mitmproxy. Compare the certificate chain, packet visibility and application data available to the proxy with the normal TLS configuration.
- Metasploit activity A - credential reuse: recover synthetic credentials from the plaintext flow and test them against the corresponding service using a suitable Metasploit auxiliary/login module or native client. Record the captured material, login outcome and the effect of one or more identity controls.
- Metasploit activity B - vulnerability exploitation: enumerate one selected Metasploitable 2 or 3 service, identify a documented vulnerability and run the corresponding Metasploit exploit with an appropriate payload. Obtain a shell or Meterpreter session, retrieve a synthetic flag, terminate the session and record the service/version, module or CVE, payload, packet evidence and relevant logs.
- Deploy the selected defenses and repeat the corresponding parts of the attack chain. The experimental set should cover at least four layers/categories and include a positioning control, a cryptographic/application control, a credential or service-hardening control, and a containment or detection control. Report the security effect, measured outcome, detection latency where relevant, false alerts, service impact and whether the next attack stage remains possible.
- Conclude with a before/after comparison that correlates PCAPs, ARP state, TLS chains, Metasploit output and network/host logs. Discuss limitations, measurement variability and any trade-offs introduced by the defenses. Offensive activity must remain inside the isolated project laboratory and use only project-owned systems, accounts and data.

Further / research references

RFC 826 — An Ethernet Address Resolution Protocol — Foundational ARP specification and basis for understanding the protocol's unauthenticated address-resolution model.

RFC 8446 — TLS 1.3 — Current TLS handshake, endpoint-authentication and protected-channel model.

RFC 6797 — HTTP Strict Transport Security (HSTS) — Browser HTTPS-enforcement policy and downgrade protection.

Georgiev, M. et al. (2012). The Most Dangerous Code in the World: Validating SSL Certificates in Non-Browser Software. ACM CCS 2012. DOI: 10.1145/2382196.2382204. — Seminal empirical analysis of certificate-validation failures and their security consequences.

MITRE ATT&CK — Adversary-in-the-Middle (T1557) — Broader threat-model context for ARP poisoning and follow-on collection/manipulation behavior.

12. Post-Quantum Network Security: TLS, PKI and Crypto-Agility

Capacity	Maximum 2 groups

Migration from quantum-vulnerable public-key cryptography to NIST PQC: hybrid TLS, PKI impact, cryptographic inventory, crypto-agility, interoperability and performance.

Context and challenge

Post-quantum cryptography (PQC) is now an engineering migration problem rather than a purely theoretical research topic. NIST standardized ML-KEM (FIPS 203), ML-DSA (FIPS 204) and SLH-DSA (FIPS 205) in 2024. Systems that depend on RSA or elliptic-curve public-key cryptography for confidentiality or authentication need an inventory and transition strategy before a cryptographically relevant quantum computer exists, especially where data has long confidentiality lifetimes ("harvest now, decrypt later").

The first difficulty is cryptographic discovery. Organizations often do not know where public-key algorithms are embedded: TLS termination, VPNs, SSH, code signing, HSMs, PKI, libraries, firmware and third-party services. The NIST NCCoE Migration to PQC project therefore emphasizes cryptographic visibility/inventory and interoperability/benchmarking. A project should begin by identifying cryptographic dependencies rather than simply compiling a PQC library.

Network protocols are moving toward hybrid transition mechanisms. RFC 10024 (2026) standardizes TLS 1.3 hybrid groups that combine ML-KEM with ECDHE, such as X25519MLKEM768. Hybrid key agreement is attractive during migration because security can survive failure of either the traditional or PQ component under the combiner assumptions, while ecosystems gain time to validate new implementations. The cost is larger handshake messages, possible fragmentation and interoperability/CPU impacts.

Signatures create a different migration problem. ML-DSA and SLH-DSA have different key/signature sizes and performance profiles from ECDSA/RSA, affecting certificates, chains, OCSP/CRLs, firmware and constrained protocols. Students should distinguish KEM migration for session key establishment from signature migration for authentication and PKI; "PQC TLS" is not a single switch.

Long-term resilience also requires crypto-agility: the ability to discover, configure, replace and retire algorithms without redesigning the whole system. NIST CSWP 39 frames agility as an operational capability across protocols, software, hardware and governance. Hard-coded algorithms, rigid certificate profiles and undocumented dependencies can make future migration more dangerous than the cryptographic primitive itself.

Assignment #2 should therefore compare a conventional TLS/PKI baseline with a PQ or hybrid configuration in a controlled environment. The goal is to measure handshake size/latency, CPU, interoperability and operational changes, then relate results to an explicit migration roadmap. It should not attempt to implement new cryptographic primitives.

The quantum threat has two different time horizons. Cryptographically relevant quantum computers do not yet exist, but data captured today can be stored and attacked later; this "harvest now, decrypt later" risk makes migration urgent for information that must remain confidential for many years. At the same time, premature replacement with immature implementations can create conventional security failures. The goal is therefore managed migration based on asset lifetime, protocol exposure and implementation readiness rather than speculation about a single quantum-computer date.

NIST's standards define distinct primitives for distinct roles. ML-KEM (FIPS 203) is a key-encapsulation mechanism used to establish shared secrets; ML-DSA (FIPS 204) and SLH-DSA (FIPS 205) are signature schemes for authentication/integrity. Students must keep these roles separate when analysing TLS, certificates, code signing and PKI. Key and signature sizes, validation cost and hardware/HSM support may differ substantially from classical algorithms and can become protocol-engineering constraints.

Hybrid deployment reduces migration risk but introduces new failure modes. RFC 10024 standardizes hybrid ECDHE+ML-KEM groups for TLS 1.3 so that connections combine a classical and post-quantum secret. This improves transition confidence, yet larger ClientHello/ServerHello flights can change fragmentation, middlebox

behavior and handshake latency. A meaningful experiment should therefore include packet sizes, retransmissions/interoperability and CPU, not merely confirm that a TLS handshake succeeds.

Crypto-agility is the organizational ability to discover where cryptography is used and replace it without redesigning every dependent system. The NIST NCCoE migration project emphasizes inventory and interoperability, while NIST CSWP 39 on crypto-agility treats replacement as a lifecycle capability. Students should identify hard-coded algorithms, certificate profiles, libraries, HSM dependencies and external services, then propose milestones for hybrid deployment, monitoring, rollback and eventual retirement of classical-only mechanisms.

The migration plan should also include failure and rollback semantics. A hybrid or PQ-enabled service may fail because a client, proxy, HSM, certificate parser or monitoring appliance does not understand new key sizes or algorithm identifiers. A safe migration therefore needs telemetry that can distinguish cryptographic negotiation failures from ordinary network failures, staged deployment, fallback policy that does not silently defeat the security objective, and an inventory that identifies which dependencies block retirement of classical-only algorithms. These operational constraints are precisely why crypto-agility is a systems property rather than a library feature.

Key concepts

- **CRQC and harvest-now-decrypt-later.** A future cryptographically relevant quantum computer threatens RSA/ECC; data captured today may be decrypted later if it must remain confidential for years.
- **KEM vs digital signature.** ML-KEM establishes shared secrets; ML-DSA/SLH-DSA authenticate/sign. Migration requirements differ across TLS, certificates and code signing.
- **Hybrid key agreement.** A classical and PQ exchange are combined so transition does not rely solely on a new primitive while deployment experience is gained.
- **Cryptographic inventory.** A machine-readable view of algorithms, certificates, libraries, endpoints and dependencies is prerequisite to prioritized migration.
- **Crypto-agility.** Ability to replace algorithms/parameters while preserving security and operations; includes architecture, configuration, governance and testing.
- **Handshake expansion.** PQ keys/ciphertexts/signatures are larger and can affect TLS record/packet sizes, fragmentation, latency and constrained links.
- **PKI migration.** Certificate profiles, CA/HSM support, trust stores, revocation and protocol libraries must evolve together; mixed classical/PQ chains may be necessary during transition.
- **Interoperability.** A cryptographically correct endpoint that cannot negotiate with the deployed ecosystem is not operationally secure; fallback and downgrade behavior must be tested.

Contextual reading

NIST Post-Quantum Cryptography program — Current status and migration guidance.

NCCoE Migration to Post-Quantum Cryptography — Cryptographic discovery, risk management and interoperability/benchmarking work.

NIST CSWP 39 — Crypto Agility — Strategies and practices for algorithm transition.

NIST IR 8547 (Initial Public Draft) — Transition to Post-Quantum Cryptography Standards — Migration-oriented NIST transition plan connecting standardized PQ algorithms to retirement of quantum-vulnerable cryptography; cite explicitly as an initial public draft.

Assignment #1 — State of the art

- Explain the quantum threat to RSA/ECC and separate confidentiality (key establishment) from authentication/signature migration. Define the data/system lifetimes that make migration urgent.
- Create a cryptographic inventory for a bounded lab/service architecture: TLS groups, certificates/signatures, CA/HSM/library dependencies and third-party components.
- Study FIPS 203/204/205 and RFC 10024. Explain hybrid TLS security goals, handshake size consequences and the rationale for maintaining a classical component during transition.
- Analyse PKI implications: certificate/signature sizes, CA support, chain validation, revocation, HSM/API compatibility and trust-store rollout.

- Apply NIST crypto-agility guidance to propose a staged migration plan with discovery, pilot, hybrid deployment, monitoring, fallback controls and eventual retirement of quantum-vulnerable algorithms.
- Define Assignment #2 milestones and metrics: handshake bytes, latency, CPU, success/failure across clients, fragmentation behavior and operational configuration complexity.

Assignment #2 — Prototype and measurement

Group allocation Maximum two groups. One group should primarily focus on hybrid TLS/key establishment and performance; the other should focus on PKI/signatures, cryptographic inventory and crypto-agility/migration engineering.

- Deploy a conventional TLS 1.3 baseline and a PQ/hybrid-capable test configuration using mature libraries/tools that support standardized algorithms. Do not implement cryptographic primitives yourself.
- Measure full and resumed handshakes where applicable: bytes on wire, connection time, CPU cost and packet/fragmentation behavior across repeated runs.
- Test interoperability with at least two client/server combinations and document negotiation/fallback behavior when PQ/hybrid support differs.
- If certificate/signature support is available, compare classical and PQ certificate/signature sizes and validation cost; otherwise model this component from published standard sizes and clearly label it as analysis rather than measurement.
- Simulate one migration failure—e.g., unsupported group/certificate, MTU constraint or disabled algorithm—and show how crypto-agile configuration/monitoring can detect and recover without silent downgrade.
- Produce a migration decision table that connects the experimental costs to the cryptographic inventory and identifies where hybrid/PQ deployment should be prioritized.

Further / research references

NIST FIPS 203 — ML-KEM standard.
NIST FIPS 204 — ML-DSA standard.
NIST FIPS 205 — SLH-DSA standard.
RFC 10024 — Hybrid ML-KEM/ECDHE groups for TLS 1.3.

13. QUIC / HTTP/3 Security and Encrypted Network Observability

Capacity	Maximum 2 groups

Modern web transport from first principles: why QUIC was introduced, how HTTP/3 maps HTTP onto it, what changes relative to HTTP/2 over TCP/TLS, and the resulting security and observability trade-offs.

Context and challenge

Students should begin by separating HTTP semantics from the transport that carries them. HTTP methods, status codes, headers and request/response semantics remain familiar in HTTP/3; the major architectural change is underneath them. HTTP/1.1 and HTTP/2 normally run over TCP (with TLS for HTTPS), whereas HTTP/3 maps those HTTP semantics onto QUIC. The MDN evolution-of-HTTP guide is a useful first introduction before the standards.

HTTP/2 explains much of the motivation for the change. It introduced binary framing and multiplexing, allowing many HTTP exchanges to share one connection, but all of those logical streams still depend on TCP's single reliable, in-order byte stream. When a TCP segment is lost, later bytes cannot be delivered until the missing data is recovered, so unrelated HTTP/2 streams may temporarily stall together. This transport-layer head-of-line blocking is one of the problems QUIC was designed to avoid by providing multiple independent streams inside the transport itself.

QUIC is therefore much more than “HTTP over UDP”. It is *a secure, connection-oriented transport protocol that uses UDP datagrams only as its substrate and implements reliability, acknowledgements, loss recovery, congestion control, flow control and multiplexed streams itself*. Moving most transport logic into user space also lets implementations evolve without waiting for new TCP behavior to reach operating-system kernels and without exposing as much transport state to middleboxes, reducing protocol ossification.

Version	Typical protocol stack	What changes for transport
HTTP/1.1	HTTP → [TLS] → TCP → IP	No application-layer multiplexing; parallelism often uses several TCP connections.
HTTP/2	HTTP/2 → TLS → TCP → IP	Multiplexed HTTP streams, but all still depend on one ordered TCP byte stream.
HTTP/3	HTTP/3 → QUIC (TLS 1.3) → UDP → IP	QUIC streams, integrated security/transport setup, path migration and stream-level loss recovery.

QUIC also integrates TLS 1.3 with transport establishment. For a fresh connection, ordinary application data is normally available after roughly one round-trip time (RTT), where one RTT is the delay for a client-to-server message and the corresponding return path. A returning client may instead resume a previous TLS session; if the server permits it, selected early application data can be sent in the client's first flight, commonly called 0-RTT. This is an optional resumption optimization that can save about one RTT, not “zero latency” and not a separate transport mode. Because 0-RTT early data can be replayed, it should be restricted to replay-safe operations or protected by appropriate application-level anti-replay controls, and the server may reject it.

Because QUIC identifies an established connection with connection IDs rather than only with one IP-address/port tuple, it can also tolerate NAT rebinding and, when policy permits, migrate across network paths. A mobile client moving from Wi-Fi to cellular connectivity may therefore preserve transport state instead of necessarily starting again from scratch. This property is useful on heterogeneous access networks but changes how firewalls, load balancers and monitoring systems associate traffic with a connection.

HTTP/3 then maps HTTP requests and responses onto these QUIC facilities. Each request/response exchange normally uses its own bidirectional QUIC stream, connection-wide control information uses dedicated streams, and QPACK provides header compression suited to multiplexed delivery. Clients negotiate HTTP/3 through TLS

ALPN and learn that an origin supports it through mechanisms such as Alt-Svc. If QUIC/UDP cannot be established, browsers can fall back to a TCP-based HTTP version, so HTTP/3 is deployed alongside HTTP/2 rather than as an all-or-nothing replacement.

These changes matter in modern networks because web applications are dominated by encrypted traffic, highly concurrent transfers, CDNs and edge services, while users increasingly move across wireless and mobile paths with variable loss and latency. QUIC can reduce setup cost, avoid connection-wide transport blocking between independent streams, preserve connections across some path changes and permit faster transport evolution. The gains are conditional rather than universal: UDP may be filtered or rate-limited, CPU and implementation quality matter, and some paths still favor mature TCP stacks. The SIGCOMM 2017 deployment paper and IMC 2017 measurement study provide useful examples of evaluating these claims empirically.

The same design decisions change the security and observability boundary. QUIC encrypts application data and much of the transport control information that is visible in TCP, improving privacy and reducing opportunities for middleboxes to ossify transport behavior. The cost is that passive network monitors have a smaller wire image from which to infer transport and application state. RFC 9312 is useful here: modern monitoring increasingly combines limited flow metadata with endpoint, server and application telemetry rather than assuming that network devices can reconstruct transport state.

QUIC's security analysis should therefore cover both protocol mechanisms and implementation behavior. Relevant mechanisms include address validation and anti-amplification limits for a UDP-based handshake, connection-ID privacy and migration, 0-RTT replay constraints, and resource/state exhaustion. At the implementation level, parsers and state machines remain attack surfaces even when the cryptographic design is sound. Recent work such as MerCuriuzz at NDSS 2026 and QUIC-Attacker at USENIX Security 2026 shows why robust implementation testing remains necessary.

Key concepts

- **HTTP semantics vs transport.** HTTP/3 retains HTTP request/response semantics; the principal architectural change is replacing the TCP transport mapping with QUIC.
- **QUIC over UDP.** UDP is the carrier. QUIC adds reliable delivery, acknowledgements, loss recovery, congestion control, flow control and connection management in user space.
- **QUIC streams.** Multiple independent ordered byte streams share one QUIC connection. Loss in one stream does not normally create connection-wide application blocking for unrelated streams.
- **Integrated TLS 1.3.** TLS authenticates peers and derives cryptographic keys as part of QUIC connection establishment; TLS records are not simply transported over a TCP-like stream.
- **Connection establishment, resumption and 0-RTT.** A fresh QUIC/TLS 1.3 connection normally needs about one RTT before ordinary application data. A returning client may use session resumption; if server policy permits, replay-safe early data can be sent in the first flight as 0-RTT. It is therefore an optional resumption optimization with weaker replay properties, not a separate transport mode.
- **Connection IDs and migration.** Connection identity is not tied only to one IP/port tuple, enabling NAT rebinding and controlled path changes such as Wi-Fi-to-cellular transitions.
- **Loss recovery and congestion control.** QUIC standardizes mechanisms around acknowledgement/loss detection while allowing congestion-control evolution; performance must be measured under realistic RTT/loss.
- **HTTP/3 mapping and QPACK.** HTTP requests/responses are mapped to QUIC streams; dedicated control streams and QPACK header compression replace parts of HTTP/2 framing behavior.
- **Protocol ossification.** Keeping most transport logic in user space and encrypting transport metadata reduces dependence on middlebox assumptions and enables faster protocol evolution.
- **Wire image and observability.** A passive observer sees less transport/application structure than with TCP-based stacks, increasing the importance of endpoint/server telemetry.
- **Address validation and amplification limits.** Before validating a client address, servers restrict how much data they send; Retry/tokens can help control spoofed-source abuse.
- **Protocol vs implementation security.** Strong cryptographic design does not eliminate parser, state-machine, resource-accounting or interoperability bugs in concrete QUIC/HTTP/3 stacks.

Contextual reading

MDN — Evolution of HTTP — Start here: concise evolution from HTTP/1.x to HTTP/2 and HTTP/3.

MDN — QUIC glossary — Short introduction to QUIC, integrated setup and stream-level loss behavior.

Daniel Stenberg — HTTP/3 Explained — Free introductory book covering why QUIC exists, QUIC streams, HTTP/3 and deployment.

Daniel Stenberg — Why QUIC? — Focused tutorial on the transport problems QUIC was designed to address.

Cloudflare Learning Center — What is HTTP/3? — Accessible operational introduction to HTTP/3, mobility, connection setup and head-of-line blocking.

Cloudflare — Introducing 0-RTT — Optional background on TLS resumption/early data, the latency saving and the replay caveat.

RFC 9204 — QPACK: Field Compression for HTTP/3. DOI: 10.17487/RFC9204 — Normative reference for HTTP/3 header compression and the design used to avoid reintroducing cross-stream head-of-line blocking.

Assignment #1 — State of the art

- Begin with a self-contained primer for a reader who knows TCP, TLS and basic HTTP but has not studied QUIC or HTTP/3. Explain the evolution HTTP/1.1 → HTTP/2 → HTTP/3 and why transport-layer head-of-line blocking motivated moving HTTP away from a single TCP byte stream.
- Explain QUIC as a transport protocol, not merely as "HTTP over UDP": streams, reliability, acknowledgements/loss recovery, congestion control, flow control, connection IDs, path validation/migration and integrated TLS 1.3. Define RTT, distinguish a fresh handshake from session resumption, and briefly explain 0-RTT as optional early data whose replay properties constrain which application operations are safe.
- Explain how HTTP/3 maps HTTP semantics onto QUIC, including per-request streams, control streams, QPACK, ALPN/endpoint discovery and fallback to TCP-based HTTP when QUIC cannot be established.
- Critically evaluate what QUIC brings to modern networks: setup latency, behavior under packet loss, mobility/NAT rebinding, user-space deployability and privacy. Use empirical research to identify where the expected benefits are conditional or workload/path dependent.
- Develop a threat model covering passive observer, active on-path attacker, spoofed-source/amplification attacker, endpoint compromise, censorship/middlebox interference and implementation bugs. Include 0-RTT replay as one concrete case: distinguish replay-safe early requests from state-changing operations that need stronger application controls.
- Analyse the QUIC wire image and identify which TCP-era monitoring/management techniques become weaker, impossible or require endpoint cooperation. Distinguish loss of passive visibility from loss of security capability.
- Review at least one recent implementation-security paper (e.g., MerCuriuzz/NDSS 2026 or QUIC-Attacker/USENIX Security 2026) and distinguish specification-level guarantees from implementation robustness.
- Produce the Assignment #2 roadmap and milestones, defining the experimental stack, HTTP/2 and HTTP/3 baselines, network-emulation conditions, packet/endpoint telemetry, migration/0-RTT tests where supported, metrics and validation criteria.

Assignment #2 — Experimental evaluation

Group allocation: Maximum two groups. One group should focus on QUIC/HTTP/3 protocol behavior, security and implementation robustness; the other should focus on encrypted network observability, monitoring/IDS implications and operational controls.

- Deploy a controlled client/server environment capable of both HTTP/2 over TCP/TLS and HTTP/3 over QUIC. Use the same application content for both protocols and collect synchronized packet captures plus endpoint/server logs.

- Use a controlled network emulator (for example Linux tc/netem) to compare several RTT/loss conditions. Measure connection establishment, request completion, multiplexed transfers and throughput across repeated runs; explain results rather than assuming HTTP/3 must always win.
- Use Wireshark or equivalent tooling to annotate a TCP+TLS/HTTP/2 connection and a QUIC/HTTP/3 connection. Identify what can be inferred by a passive observer at each phase and which information is only available at the endpoints.
- Optional resumption experiment: where the selected implementation supports it, establish resumption state and compare a resumed connection with and without 0-RTT using a replay-safe request. Use captures/logs to show early data in the first flight and explain server rejection and replay implications. Keep any replay experiment inside the laboratory.
- Exercise a NAT-rebinding or path-change scenario and correlate packet captures with QUIC connection IDs and endpoint logs. Explain whether the connection survives and which security/operational checks are triggered.
- Evaluate one observability strategy for encrypted QUIC traffic—flow metadata, endpoint telemetry, eBPF/runtime events, server logs or exported metrics—and compare its detection context with passive TCP/TLS inspection.
- Include one controlled robustness experiment, such as safe protocol fuzzing against a local QUIC implementation or a bounded stream/resource stress test. Report limits and failures without conducting denial-of-service tests against public services.
- Conclude with a technically justified assessment of where HTTP/3 is advantageous, where HTTP/2/TCP remains competitive or necessary, and how the security/observability trade-off should influence deployment.

Further / research references

RFC 9000 — QUIC: A UDP-Based Multiplexed and Secure Transport. DOI: 10.17487/RFC9000 — Core QUIC transport specification.

RFC 9001 — Using TLS to Secure QUIC. DOI: 10.17487/RFC9001 — TLS integration, packet protection and 0-RTT security considerations.

RFC 9114 — HTTP/3. DOI: 10.17487/RFC9114 — Mapping of HTTP semantics onto QUIC.

RFC 9312 — Manageability of QUIC. DOI: 10.17487/RFC9312 — Wire image, middleboxes and operational visibility.

Langley, A. et al. (2017). “The QUIC Transport Protocol: Design and Internet-Scale Deployment.” ACM SIGCOMM 2017. DOI: 10.1145/3098822.3098842 — Foundational large-scale design/deployment paper.

Kakhki, A. M. et al. (2017). “Taking a Long Look at QUIC.” ACM Internet Measurement Conference (IMC 2017). DOI: 10.1145/3131365.3131368 — Empirical evaluation methodology and performance analysis.

Smith, J.-P., Dolfi, L., Mittal, P., & Perrig, A. (2022). “QCSD: A QUIC Client-Side Website-Fingerprinting Defence Framework.” USENIX Security 2022, pp. 771–789. USENIX paper page — Example of privacy/security research built on QUIC/HTTP/3.

Zohaib, A. et al. (2025). “Exposing and Circumventing SNI-based QUIC Censorship of the Great Firewall of China.” USENIX Security 2025, pp. 783–802. USENIX paper page — Contemporary measurement of QUIC filtering/censorship and encrypted-handshake processing.

Wang, K. et al. (2026). “Identifying Logical Vulnerabilities in QUIC Implementations.” NDSS 2026. DOI: 10.14722/ndss.2026.231777 — MerCuriuzz black-box fuzzing and logical implementation vulnerabilities.

Erinola, N., Maehren, M., Brinkmann, M., & Schwenk, J. (2026). “Breaking the Boundaries: Analyzing QUIC Frame-Packet Interactions With QUIC-Attacker.” USENIX Security 2026, pp. 341–358. USENIX paper page — Implementation robustness across QUIC frame/packet interactions.

14. WireGuard Overlay Networks: Tailscale vs NetBird

Capacity	Maximum 2 groups

Comparative study of identity-aware WireGuard overlays: cryptographic data plane, key/policy orchestration, NAT traversal, control-plane trust and Tailscale vs NetBird operational trade-offs.

Context and challenge

VPNs create protected connectivity across untrusted networks, but classical remote-access/site-to-site designs are often gateway-centric: clients terminate at concentrators, administrators manage routes and certificates centrally, and traffic may hairpin through a small number of gateways. Modern secure overlays instead try to give each endpoint an authenticated network identity and establish direct encrypted paths where possible. This shifts the security problem from "who can reach the VPN gateway?" toward continuous identity, key distribution, policy and endpoint lifecycle.

WireGuard provides a deliberately small data-plane protocol. The original NDSS 2017 WireGuard paper describes Cryptokey Routing, a NoiseIK-derived handshake, fixed modern cryptographic primitives and roaming-friendly UDP endpoints. The mechanised proof by Lipp, Blanchet & Bhargavan is important for separating formally analysed protocol properties from the security of a complete deployed overlay.

WireGuard follows a minimalist design that contrasts with more configurable VPN systems such as OpenVPN. OpenVPN uses a TLS-based control channel, commonly with certificate/PKI management and extensive authentication/algorithm choices. This flexibility supports heterogeneous deployments but increases configuration and negotiation surface. WireGuard fixes a small cryptographic suite and identifies peers by public keys, substantially simplifying the tunnel protocol and its auditability. The trade-off is that WireGuard deliberately does not solve user identity, key distribution/revocation, policy, discovery, DNS, NAT traversal or fleet lifecycle.

Tailscale and NetBird address those missing operational functions by acting as control-plane/key-orchestration systems around a WireGuard data plane. Tailscale's control/data-plane documentation explicitly separates coordination (identity, peer/key and policy distribution) from endpoint-to-endpoint encrypted traffic. NetBird's architecture similarly separates client, Management, Signal and Relay components and supports direct peer-to-peer WireGuard connectivity whenever NAT traversal succeeds.

The control plane becomes security-critical even when it never receives plaintext data-plane traffic. A compromised coordinator that can substitute public keys or authorize an attacker-controlled node can undermine endpoint trust. Tailnet Lock is especially valuable academically because it explicitly addresses this residual trust by requiring user-controlled signatures over node-key distribution. NetBird offers a different trust/operations option through self-hosting of management infrastructure.

NAT traversal and relays are another important dimension. Endpoints discover candidate addresses and attempt direct UDP paths; if direct connectivity fails, relays forward already-encrypted WireGuard traffic. Students should measure the direct/relayed distinction because it affects latency, throughput, failure modes and trust/metadata exposure even though confidentiality remains end-to-end.

The comparison should therefore be scientific rather than a feature checklist: trace enrollment, key lifecycle, policy calculation, path establishment, relay fallback and revocation; define the trust placed in identity provider, control plane, relay and endpoints; then measure convergence and data-plane performance under identical conditions. Relate the result to NIST SP 800-207 Zero Trust Architecture without using "Zero Trust" as a product label.

Identity enrollment is the first place where the overlay platform adds semantics beyond raw WireGuard. A node must be associated with a user, device or setup credential before its public key can be distributed to authorized peers. That makes the identity provider, enrollment token and control plane part of the security boundary even though WireGuard private keys remain local. Students should trace this lifecycle explicitly for both products and distinguish compromise of an endpoint key from compromise of an account or control-plane authorization.

Revocation illustrates why key orchestration matters. Base WireGuard peers are configured with public keys and AllowedIPs but do not provide a global directory or enterprise revocation service. Platforms must remove or replace node keys, update peer maps and converge policy across online endpoints. Tailscale's control/data-plane model and NetBird's architecture should be compared in terms of what state is centrally authoritative, what remains cached during a control-plane outage, and how quickly a revoked peer actually loses connectivity.

NAT traversal is also a control-plane-assisted problem. Peers discover candidate endpoints and attempt direct UDP paths; if these fail, a relay carries already end-to-end encrypted WireGuard traffic. The relay therefore affects metadata, availability and performance even if it cannot decrypt payloads. Students should distinguish Tailscale's relay mechanisms from NetBird's Signal/Relay components and experimentally force direct and relayed paths so that "peer-to-peer" is measured rather than assumed.

Self-hosting changes who operates the control plane, not the underlying need to secure it. NetBird's self-hosting guidance moves identity integration, upgrades, availability, backup and coordinator protection to the organization. Tailscale's Tailnet Lock takes a different approach by cryptographically constraining node-key distribution even if the coordination service is compromised. These designs create an excellent trust-model comparison: managed service vs self-operated control plane vs user-controlled signatures, each with different operational costs and residual risks.

Key concepts

- **WireGuard peer identity / Cryptokey Routing.** A peer public key is associated with allowed IP prefixes, combining tunnel identity and routing authorization at the data plane.
- **Data plane.** Endpoint WireGuard sessions encrypt actual user traffic; direct peer-to-peer paths are preferred and private keys remain on endpoints.
- **Control plane.** Authenticates/enrolls devices, distributes public keys and peer state, computes policy, coordinates discovery and may integrate an external identity provider.
- **Key orchestration.** The overlay binds WireGuard node keys to users/devices, distributes only authorized keys, rotates/revokes membership and turns high-level policy into endpoint filters.
- **NAT traversal.** Endpoint discovery, UDP hole punching and connectivity checks attempt direct paths; relays provide fallback when network policy/NAT prevents peer-to-peer communication.
- **Relay trust.** A relay can observe metadata and availability but should see only WireGuard-encrypted payloads; compromise should not grant plaintext access if endpoint keys remain secure.
- **Control-plane trust.** Incorrect key/policy distribution can authorize an attacker even though WireGuard cryptography is sound; Tailnet Lock and self-hosting represent different responses to this trust.
- **Zero Trust relation.** Identity-aware least-privilege connectivity aligns with ZTA principles, but Zero Trust also includes resource policy, device posture, continuous evaluation and governance beyond the overlay tunnel.

Contextual reading

NIST SP 800-207 — Zero Trust Architecture — Authoritative model for identity/resource-centric trust decisions.

Tailscale — Control and data planes — Current architecture and control/data plane responsibilities.

Tailscale — Tailnet Lock white paper — Detailed analysis of residual control-plane trust and signed node-key distribution.

NetBird — How NetBird Works — Management, Signal, Relay and WireGuard data-plane architecture.

OpenVPN 2.6 manual — Technical baseline for TLS/PKI-based VPN comparison.

WireGuard — Protocol & Cryptography — Official concise description of the WireGuard handshake, cryptographic construction and transport behavior before studying overlay control planes.

Assignment #1 — State of the art and comparative architecture

- Contextualize remote-access, site-to-site and mesh/overlay VPN designs. Explain why identity-aware peer-to-peer overlays change routing, trust and key-management assumptions.
- Explain WireGuard at protocol level: interfaces/peers, Cryptokey Routing, handshake and key rotation, fixed primitive suite, UDP transport and roaming. Distinguish protocol guarantees from platform-level identity/policy features.

- Compare WireGuard and OpenVPN scientifically in protocol/configuration model, PKI/algorithm negotiation, attack/configuration surface and performance evidence; avoid treating either as universally “more secure.”
- Decompose Tailscale and NetBird into control/data planes and trace enrollment, node-key distribution, IdP integration, ACL/policy calculation, DNS/routing, NAT traversal, relay fallback and revocation.
- Build a threat model for endpoint compromise, stolen enrollment credential, compromised IdP, malicious/compromised control plane, malicious relay, stale keys and policy misconfiguration. Include Tailnet Lock and NetBird self-hosting in the trust comparison.
- Produce the Assignment #2 roadmap and fairness plan: identical endpoint topology, services, access intent, direct/relayed conditions, lifecycle tests, metrics and version-pinned configurations.

Assignment #2 — Comparative deployment and experimental evaluation

Group allocation Maximum two groups. One group should emphasize security architecture, control-plane trust, identity/key lifecycle and self-hosting; the other should emphasize network behaviour, NAT traversal/relaying, performance and operational manageability. Both groups must perform a direct Tailscale-versus-NetBird comparison.

- Deploy the same three-or-more-endpoint topology with Tailscale and NetBird across at least two underlying networks/NAT domains, using equivalent services and access-policy intent.
- Document enrollment and key orchestration without exposing private keys: identity authentication, node public-key distribution, peer visibility, policy application and revocation.
- Use packet capture and platform diagnostics to identify direct versus relayed data paths, verify that relay traffic remains WireGuard-encrypted and explain why fallback occurred.
- Benchmark connection establishment, RTT and TCP/UDP throughput for direct paths and, where feasible, relayed paths. Record CPU/resource overhead and repeat measurements.
- Evaluate at least two lifecycle/failure scenarios: peer revocation, policy update, control-plane outage after convergence, credential expiry/rotation or endpoint movement between networks. Measure convergence/recovery time and cached-state behavior.
- Produce an evidence-based comparison matrix for at least two deployment scenarios (e.g., managed multi-device organization vs self-hosted infrastructure), separating security/trust, operations, performance and lock-in considerations.

Further / research references

Donenfeld (2017), WireGuard: Next Generation Kernel Network Tunnel — NDSS foundational paper.

Lipp, Blanchet & Bhargavan (2019), A Mechanised Cryptographic Proof of the WireGuard Virtual Private Network Protocol — EuroS&P 2019 formal security analysis. DOI: 10.1109/EuroSP.2019.00026.

Mackey, S., Mihov, I., Nosenko, A., Vega, F., & Cheng, Y. (2020). A Performance Comparison of WireGuard and OpenVPN. CODASPY 2020. DOI: 10.1145/3374664.3379532.

Rules for project submission and evaluation

- Projects are completed in groups of three students unless an exception is approved by the instructor.
- Topic allocation limits are indicated at the start of each theme. Where multiple groups share a broad topic, they must use distinct subproblems, systems or anchor papers as specified.
- Assignment #1 is a technical state-of-the-art report. It should synthesize and compare research rather than concatenate summaries of sources.
- Assignment #2 extends the same topic with a reproducible prototype, experiment or measurement study and must explicitly connect results back to the research questions from Assignment #1.
- Reports must identify software versions, configurations, datasets and experimental assumptions. Source code/configuration/scripts must include a README with reproduction instructions.
- All experiments involving offensive techniques must be authorized and restricted to controlled project infrastructure.
- Submission and presentation dates are published in Moodle.

Recommended report structure

- Problem statement and motivation.
- Threat model / security assumptions.
- Related work and critical comparison of approaches.
- Experimental methodology or prototype architecture.
- Results with appropriate metrics and uncertainty/limitations.
- Security analysis and countermeasures.
- Reproducibility information.
- Ethical/legal considerations where relevant.
- References using a consistent academic citation style.

Submission packaging

Unless Moodle states otherwise, use the established naming convention where X is the TP class and Y the group number:

- **TPX_groupY.zip —** submission archive.
- **code/ —** relevant code, scripts, configuration files and small reproducibility artefacts.
- **README —** clear setup and reproduction instructions.
- **TPX_groupY_report_1.pdf —** Assignment #1 report.
- **TPX_groupY_report_2.pdf —** Assignment #2 report.

Large artefacts Do not place large VM images, raw datasets or very large packet captures in the submission archive unless explicitly requested. Provide a stable retrieval location and integrity hash where appropriate.